

Hybrid MPI+OpenMP Performance of PETSc Krylov Solvers on ARCHER2

Leyan Li

MSc in High Performance Computing
EPCC, University of Edinburgh

August 2026

A dissertation submitted in partial fulfilment
of the requirements for the degree of
Master of Science.

Abstract

PETSc can combine MPI ranks with OpenMP threads, but the best balance depends on solver work, communication, and the decomposition. This dissertation measures four full-node layouts on ARCHER2: 128 MPI ranks with one thread per rank, and hybrid layouts with two, four, or eight threads per rank. Matched two- and three-dimensional Poisson benchmarks cover six problem sizes from 5.0 to 164.6 million unknowns on 1–32 nodes. Each formal configuration has three runs. One warm-up solve builds the GAMG hierarchy; 19 timed solves then reuse it in a dedicated PETSc logging stage. The unknowns-per-core admission filter orders the 11 admitted size–node pairs into two weak-scaling chains, along which the layout comparison is made at constant work per core.

The two dimensions differ in the shape of the result, not only in which layout is fastest. In 2D the layout effect is ordered by concurrency: every hybrid layout improves relative to flat MPI as the node count grows, monotonically in four of the six measured series. 64×2 is faster per iteration than flat MPI at all 11 pairs, by 4.3% at one node and 12.8% at 16, and reaches 13.1% higher raw throughput at 164.56 million unknowns on 32 nodes; 32×4 crosses from 9.6% slower at one node to 6.3% faster at 32. In 3D the advantage is larger, reaching 33.2%, but no series is monotone and the preferred layout changes with scale and node count. Eight threads per rank wins no point in either dimension at constant work per core.

A matched extension holds the problem size fixed near 20 million unknowns and scales from one to 32 nodes, and there the layouts differ in where they saturate rather than only in speed. The 2D curves stay within a factor of 1.81 of each other and all four are still improving at 4096 cores, whereas in 3D every layout saturates and turns upward at a node count that rises with its thread count: eight nodes for flat MPI, which is $2.6\times$ slower again by 32, and beyond 32 for 16×8 , which ends $3.81\times$ faster than it. What distinguishes the layouts under strong scaling is where they stop scaling, so the same layout is both the worst choice at comfortable occupancy and the best one past saturation. PETSc event data show hybridisation reducing exposure to halo exchange and global reduction, and eleven Linaro MAP profiles measure the cost that repays: the share of core-time spent waiting in the OpenMP runtime rises with threads per rank, reproducibly across two builds, and the linked threaded BLAS library accounts for at most 2.1% of solve-window core-time. The hardware mechanism behind the waiting is not established here.

Acknowledgements

Contents

Abstract	i
Acknowledgements	ii
1 Introduction	1
1.1 Motivation	1
1.2 Aims and Research Questions	1
1.3 Contributions	2
1.4 Dissertation Structure	3
2 Background and Related Work	4
2.1 Parallel Programming Models	4
2.2 Limitations of Flat MPI	5
2.3 Hybrid MPI+OpenMP	6
2.4 ARCHER2 Node Architecture and NUMA	6
2.5 PETSc and Krylov Solvers	8
2.6 Performance Analysis Tools	9
2.7 Summary and Research Gap	9
3 Experimental Platform and Methodology	10
3.1 Hardware and Software Environment	10
3.2 Benchmark Problems and Drivers	10
3.3 Solver Configuration	11
3.4 Configuration Space and Selection Rules	12
3.5 Bounding the Thread Count	12
3.5.1 Exploratory Single-Node Sweep	12
3.5.2 The Same Bound on the Production Configuration	14
3.6 Job Configuration and Thread Placement	15
3.7 Metrics	15
3.8 Superseded Campaigns	15
3.9 Reproducibility	17
4 2D and 3D Problems	18
4.1 Experimental Setup	18
4.1.1 Weak-Scaling Chains	19
4.1.2 Strong-Scaling Design	19
4.2 Weak Scaling at Constant Work per Core	19
4.2.1 2D	20
4.2.2 3D	21
4.2.3 Layout Ranking and Efficiency	22
4.3 Strong Scaling at Fixed Problem Size	24
4.3.1 2D	24

4.3.2	3D	25
4.3.3	Endpoints	27
4.4	Interpretation	27
5	Profiling and Bottleneck Analysis	29
5.1	Why a Repeated-Solve Driver	29
5.2	Analysis Methodology	29
5.3	MAP Profiling Matrix Results	29
5.4	Bottleneck Identification	31
5.4.1	What is Measured	32
5.4.2	Halo Exchange Dominates, and Hybridisation Reduces It	32
5.4.3	Reduced Exposure is Necessary but not Sufficient	33
5.4.4	What This Does and Does Not Establish	35
6	Discussion	36
6.1	Answering the Research Questions	36
6.1.1	RQ1: Does the threaded backend reduce time to solution, and by how much?	36
6.1.2	RQ2: At which balance, and how does the answer change with the regime?	36
6.1.3	RQ3: Which parts of the solve account for the difference?	37
6.1.4	RQ4: How far do the trends generalise, and what would establish a mechanism?	37
6.2	Threats to Validity	38
6.3	Practical Guidance for PETSc on ARCHER2	38
7	Conclusion and Future Work	40
7.1	Conclusions	40
7.2	Future Work	41
	Bibliography	42
A	Reproducibility: Repository and Scripts	43
A.1	Layout	43
A.2	Evidence Index	44
A.3	Regenerating the results	45
B	Supplementary Figures	47
C	Full log_view Excerpts	49

List of Figures

2.1	Hierarchy of an ARCHER2 compute node. Each node has two 64-core AMD EPYC 7742 processors, eight 16-core NUMA regions, and four-core CCXs that each share 16 MB of L3 cache. Source: ARCHER2 hardware documentation [8]. .	7
3.1	Single-node default-solver sweep on a 1000×1000 grid. Adding OpenMP threads to one MPI rank leaves the cost per iteration almost unchanged. Adding MPI ranks at one thread each reduces it by two orders of magnitude. Both lines start from the same one-rank, one-thread run.	13
4.1	2D time per KSP iteration relative to the 128×1 baseline at the same size-node pair; values below 1 mean the hybrid layout is faster. Whiskers propagate the observed range of three formal runs.	20
4.2	Median 2D time per KSP iteration along the two unknowns/core chains. A horizontal line would be ideal weak scaling.	21
4.3	3D time per KSP iteration relative to the 128×1 baseline at the same size-node pair. Unlike the 2D case in Figure 4.1, no series is monotone.	21
4.4	Median 3D time per KSP iteration along the two unknowns/core chains.	22
4.5	2D time per solve at a fixed 20.25M unknowns, one to 32 nodes. Both axes are logarithmic, so ideal strong scaling is a straight line of slope -1 . Whiskers show the observed range of three formal runs.	25
4.6	3D time per solve at a fixed 20.35M unknowns, one to 32 nodes. Flat MPI and 64×2 reach a minimum and then get slower, while the two most-threaded layouts continue to improve.	26
4.7	3D parallel efficiency at a fixed 20.35M unknowns, each layout against its own one-node time. Because the baselines differ, this figure ranks scaling behaviour and not speed; flat MPI falls to 10.9% at 32 nodes while 16×8 retains 44.5%. . .	26
5.1	Where the cores are during the solve, from the MAP call trees. Within each panel the OpenMP waiting share grows and the MPI share shrinks as threads replace ranks, and both effects strengthen with node count. Paired bars are the two PETSc builds at the same configuration.	31
5.2	2D share of <code>RepeatedSolves</code> <code>KSPSolve</code> time spent inside halo exchange and global reduction events. The ordering matches the time-per-iteration ordering in Figure 4.1 at every node count.	32
5.3	3D share of <code>RepeatedSolves</code> <code>KSPSolve</code> time spent inside halo exchange and global reduction events. Note the vertical scale: flat MPI reaches 84.6% at the largest point.	33
5.4	3D exposure removed relative to flat MPI against the resulting time per iteration, one point per size-node pair. Where a layout's fitted line crosses parity is the break-even column of Table 5.3; the lines are descriptive, not predictive.	34

5.5	2D exposure removed relative to flat MPI against the resulting time per iteration. Compare with Figure 5.4: 64×2 and 32×4 descend as they do in 3D, but 16×8 sits well above parity at every point with no slope at all.	35
B.1	Median 2D repeated-solve throughput along the two unknowns/core chains. Whiskers show the observed range of three formal runs. All layouts use 128 cores per node, so a fixed node count means a fixed core count.	47
B.2	Median 3D repeated-solve throughput along the two unknowns/core chains. Whiskers show the observed range of three formal runs.	48

List of Tables

2.1	The three execution models as they apply within one ARCHER2 node. The hybrid row is the subject of this dissertation. It inherits both cost columns.	5
2.2	The four fully populated layouts against the ARCHER2 node hierarchy, with close binding. Every layout uses all 128 physical cores. What changes is the number of subdomains and where each rank's threads sit relative to the shared-L3 and NUMA boundaries. The counts are arithmetic from the node topology measured above; they describe placement, not a measured claim about cause.	7
2.3	Operations in one GAMG-preconditioned CG iteration, with the PETSc event that records each and the number of calls per outer iteration measured in a representative run. Neighbour and global costs scale differently with the rank count, which is why Section 5.4 separates them.	8
3.1	Final iteration count of the single-node default-solver sweep on a 1000×1000 grid, for every ranks $\times$ threads ≤ 128 combination. Every row is constant: the count is fixed by the rank count alone. The 128-rank configuration reached the 10,000-iteration limit without converging.	11
3.2	Cost per KSP iteration for the eight fully populated single-node layouts (ranks $\times$ threads = 128) on a 1000×1000 grid. The trend is monotone in the rank count, with no threshold at any particular thread count. The 128×1 figure is measured over its non-converged 10,000 iterations, which still gives a valid per-iteration cost.	14
3.3	Time per KSP iteration (ms) for the five full-node layouts of the multi-node production campaign, one run per cell, medians not available. Every layout uses all 128 cores of every node; only the rank-thread split differs. Each row is monotone from left to right, which is the property the $\{1, 2, 4, 8\}$ restriction rests on. The campaign has no 128×1 cell, and most rows sit below the 25,000 unknowns/core admission window, so this table bounds the configuration space rather than ranking the admitted layouts.	14
3.4	Relative tolerance actually used by <code>ex2</code> on a 100×100 grid, read from <code>-ksp_view</code> , for the three option styles. The double dash is silently ignored and the mesh-dependent default survives; <code>-options_left</code> names the unconsumed option explicitly.	16
4.1	Grids and node counts of the six scales. The weak-scaling column is the set admitted by the unknowns-per-core filter of Section 3.4; the last column is the fixed-size campaign of Section 4.1.2, which runs the 20m problem over the whole node range and is deliberately not limited by that filter.	18
4.2	The 11 admitted size-node pairs, ordered into two weak-scaling chains. Each row holds unknowns per core near-constant while the node count doubles, so a layout's cost along a row isolates the effect of concurrency at fixed local work.	19

4.3	Fastest full-node layout at every admitted size–node pair, by median raw throughput over three formal repeats, with its margin over flat MPI at the same core count. A margin of 0.0% means flat MPI itself was fastest. The 2D 41m/8-node row is a tie rather than a win: the two medians differ by 0.4% and their observed ranges overlap. Throughput folds in the GAMG iteration count, which is why the last column is given.	23
4.4	Weak-scaling efficiency (%) at the largest node count of each chain, relative to the same layout’s one-node time. Each layout is measured against itself, so this quantifies degradation, not absolute speed.	23
4.5	Fastest full-node layout at each node count for the fixed 20M problems, with its median time per solve and its advantage over flat MPI at the same core count. Medians of three runs; the 32-node row has about 4.9 thousand unknowns per core.	27
4.6	The 32-node endpoint of the fixed-size curves. Speedup and cumulative efficiency are measured against the same layout’s one-node time, so they reward a poor baseline; the doubling efficiency is the 16-to-32-node step alone. Time per solve is the only column that compares layouts directly.	27
4.7	The two dimensions on the quantities this chapter compares them by. The upper block is the weak-scaling matrix at constant work per core, the lower block the fixed-size curves of Section 4.1.2. Throughput gain and time saving are different conventions and are both given, because the two dimensions are quoted in both.	28
5.1	Per-event time and flop shares, RepeatedSolves stage, 20.25M, 16×8	29
5.2	Share of total core-time inside the solve window, by the library owning the executing frame, for the eleven usable MAP profiles at 20.25M unknowns in 2D. OpenMP is the GNU runtime’s spin and wait paths, that is threads with no work. MAP instrumentation inflates these runs, so the ordering between layouts is the result. The absolute values are not transferable to the uninstrumented campaigns.	30
5.3	Reduction in synchronisation exposure relative to flat MPI at the same size–node point, against time per iteration relative to flat MPI. The break-even column is where the fitted line crosses parity, i.e. how much exposure a layout must remove before its thread overhead is repaid. Eleven points per layout; the fits are descriptive.	33
A.1	Repository layout. Raw logs and retrieved job CSVs are immutable evidence; everything under a <i>derived</i> heading is regenerated by the scripts.	43
A.2	Evidence index. Script paths are relative to analysis/scripts/ , derived and raw paths to analysis/ . Log counts are the files present, which for the 3D fixed-size campaign exceeds the 72 accepted runs because superseded job attempts are retained. The last two rows are read directly from a named file and have no derived stage.	44

Chapter 1

Introduction

1.1 Motivation

Sparse iterative solvers dominate the run time of a large class of scientific applications, and PETSc [1, 2] is one of the most widely used libraries for writing them. PETSc has always parallelised across distributed memory with MPI. Since version 3.22 it can also be configured with threaded kernels, so a fixed allocation of cores can be spent on MPI ranks, on OpenMP threads inside each rank, or on any combination of the two [3].

That choice is not obviously free. An ARCHER2 compute node provides 128 physical cores across two sockets, eight NUMA regions, and thirty-two four-core shared-L3 domains. Flat MPI exposes every subdomain as a separate process with its own address space, message matching, and solver metadata. Replacing ranks with threads reduces the number of MPI endpoints and of subdomain boundaries, but requires the runtime to create, schedule, and synchronise thread teams, and makes several cores compete for the same caches and memory channels. Both effects grow with the thread count, in opposite directions.

Neither effect can be predicted from the core count. The saving depends on how much of an iteration is spent communicating, which depends in turn on the problem size, the stencil, the preconditioner, and the number of nodes. The cost depends on how much work each threaded kernel exposes and on where the threads sit in the memory hierarchy. A user allocating a full ARCHER2 node has four sensible layouts available and no principled way to choose between them. The purpose of this dissertation is to measure that choice.

1.2 Aims and Research Questions

This study measures PETSc rank–thread configurations for matched two- and three-dimensional Poisson problems on ARCHER2. It compares flat MPI with three hybrid MPI+OpenMP layouts across six problem sizes from 5.0 to 164.6 million unknowns and 1–32 fully populated nodes, holding the core count, the solver, and the convergence criteria fixed so that only the layout varies. Because the admission rule that keeps the work per core constant also limits each problem size to one or two node counts, a matched extension repeats the same contract on a fixed problem of about 20 million unknowns over the whole node range, so that the two standard scaling regimes are both covered. Four questions follow the feasibility study that preceded this work, narrowed to what the completed campaigns can answer.

RQ1 Does PETSc’s threaded-kernel backend reduce the time to solution relative to flat MPI on a fully populated node, and by how much?

RQ2 If it does, at which rank–thread balances, and how does the answer change with problem size, node count, dimensionality, and scaling regime?

RQ3 Which parts of the solve account for the difference?

RQ4 How far do the observed trends generalise, and what would be needed to establish a mechanism for them?

RQ1 and RQ2 are answered by Chapter 4, RQ3 by Chapter 5, and RQ4 by Chapter 6. The feasibility study’s hypotheses expected a benefit at two to four threads per rank and expected `VecMDot` and `PCApply` to be the limiting kernels. The first expectation is borne out. The second was formed while the study still used PETSc’s default GMRES solver. The change of solver described in Section 3.3 replaces `VecMDot` with `VecTDot` and moves the dominant cost elsewhere.

1.3 Contributions

1. **A matched 2D and 3D rank–thread study at full node occupancy.** Four layouts, six problem sizes, 1–32 nodes, three formal repeats, and 264 content-validated formal runs, with a further 144 runs on matched fixed-size problems. Every log is checked against the intended build, the requested rank and thread counts, the expected number of converged solves, and the linked library, and the parsed results are reconciled against the independently retrieved job CSVs.
2. **Evidence that the layout ranking depends on concurrency, not just on the workload.** Read as weak scaling at fixed unknowns per core, the 2D advantage of hybrid execution grows monotonically with node count: 64×2 is faster per iteration than flat MPI at all 11 size–node points, by 4.3% at one node and 12.8% at 16, and 32×4 crosses from 9.6% slower at one node to 6.3% faster at 32. A layout comparison performed at small node counts does not transfer to large ones.
3. **Evidence that the layout ranking also depends on the scaling regime.** On matched fixed-size problems near 20 million unknowns, scaled from one to 32 nodes, the 2D layouts remain within a factor of 1.81 of each other and all keep scaling, while the 3D layouts diverge to a factor of 3.81 because each saturates and turns upward at a node count that rises with its thread count: eight nodes for flat MPI, 16 for the two intermediate layouts, and beyond 32 for 16×8 . 16×8 is last at constant work per core and first at 4096 cores on a fixed problem, so the same measurement supports opposite advice in the two regimes.
4. **A quantitative account of why, from both sides of the trade.** PETSc’s own event data show that hybridisation reduces the share of the solve exposed to halo exchange and global reduction at 21 of 22 size–node points, and that within each layout the size of that reduction correlates with the speed advantage. Eleven Linaro MAP profiles, parsed directly from the profile files, measure the countervailing cost as time spent waiting in the OpenMP runtime, which rises with threads per rank while the MPI share falls, reproducibly across two PETSc builds. Each doubling of threads per rank removes more exposure but must remove more before it repays that cost, which explains the two-to-four-thread optimum, the non-monotone 3D ordering, and why 16×8 is slowest in 2D despite removing the most exposure.
5. **A repeated-solve measurement method.** A driver that pays the GAMG setup once and then times 19 solves in a dedicated PETSc logging stage, so the reported figures are steady-state solve costs rather than a mixture of hierarchy construction and solve. The same design is used for the scaling results and for the profiling analysis.
6. **A reproducible pipeline and an explicit account of what the evidence does not support.** Raw logs, analysis scripts, and figure generation are version-controlled and re-runnable from one command per campaign, including the profiler analysis, which is

computed from the MAP files themselves rather than from a GUI export. Section 6.2 states the limits, including why a rank-dependent preconditioner and a silently ignored tolerance option together make the earlier default-solver comparison ill-posed, and why no hardware mechanism is claimed.

1.4 Dissertation Structure

Chapter 2 introduces the execution models, the ARCHER2 node hierarchy, CG with algebraic multigrid, and the two profiling tools, and identifies the gap this work addresses. Chapter 3 defines the drivers, the solver configuration and why it was changed, the admissible configuration space, thread placement, and the metrics. Chapter 4 presents the validated 2D and 3D results as weak scaling along two unknowns-per-core chains, then as fixed-size strong scaling from one to 32 nodes. Chapter 5 explains the measurement design and gives the profiler and event-level bottleneck analysis. Chapter 6 answers the research questions, states the threats to validity, and gives practical guidance, before Chapter 7 summarises the findings and the work that remains.

Chapter 2

Background and Related Work

2.1 Parallel Programming Models

PETSc applications run on machines whose memory is distributed across nodes and non-uniform within each node. The programming model determines which of these boundaries the application exposes explicitly.

MPI uses independent processes with private address spaces. Processes exchange data through point-to-point operations and collectives defined by the MPI standard [4]. This model matches a distributed-memory machine: each rank owns part of the problem, and communication makes remote dependencies explicit. MPI implementations also run within a node, often through shared-memory transports, so “MPI-only” does not imply that every message crosses the network. It does retain process-level decomposition, message matching, and per-rank library state.

OpenMP uses a shared-memory, fork-join execution model. Threads in a team can access the same address space, while worksharing constructs divide loop iterations among them [5]. OpenMP removes the need to express intra-process data exchange as messages, but the runtime must create or wake thread teams, distribute work, and synchronise them. Threads can also contend for cache capacity and memory bandwidth. A threaded loop is therefore faster only when it exposes enough work to offset these costs.

A hybrid program assigns one or more MPI ranks to each node and creates an OpenMP team inside every rank. MPI still communicates between subdomains. OpenMP processes the data owned by each rank. The rank-thread product gives the number of cores used, but two layouts with the same product are not equivalent. For example, 128×1 creates 128 subdomains and MPI endpoints on an ARCHER2 node, whereas 32×4 creates 32 larger subdomains and four threads per subdomain. The latter can reduce rank-local metadata and the number of halo interfaces, but adds thread scheduling and shared-resource contention. Table 2.1 summarises what each model makes explicit and what it therefore costs.

Partitioned Global Address Space models such as OpenSHMEM and Chapel express locality differently again, but PETSc uses MPI for distributed-memory communication, so the comparison here is between flat MPI and PETSc’s hybrid path rather than between programming-model families.

Table 2.1: The three execution models as they apply within one ARCHER2 node. The hybrid row is the subject of this dissertation. It inherits both cost columns.

Model	Memory	Data exchange	Principal cost
MPI	Private per rank	Explicit messages	Message matching, per-rank solver state, one subdomain boundary per rank
OpenMP	Shared within a team	Shared addresses	Team creation, synchronisation, cache and bandwidth contention
Hybrid	Shared inside a rank, private between ranks	Messages between ranks only	Both, traded against each other by the rank-thread balance

2.2 Limitations of Flat MPI

Flat MPI assigns one rank to each core. Its main advantage is uniformity: the same domain decomposition and communication model applies within and between nodes, and each rank executes single-threaded code. This often gives predictable performance and avoids thread safety and OpenMP scheduling costs. A hybrid configuration must outperform this strong baseline, not an abstract MPI implementation.

The costs of flat MPI become visible under either of the two standard scaling regimes, and this dissertation uses both, so the distinction is worth stating precisely. Under *strong* scaling the global problem size is fixed while the core count grows, so the work per core falls. Under *weak* scaling the problem size grows with the core count so that the work per core stays constant, and perfect weak scaling would hold the time per iteration constant. Strong scaling measures how far a fixed problem can be spread. Weak scaling measures what concurrency costs once the local work is held still. The unknowns-per-core admission rule in Section 3.4 makes the second regime the natural reading of the campaign, and Section 4.1.1 sets it out.

Under strong scaling, each rank receives less local work, but latency-sensitive operations do not shrink at the same rate. A stencil matrix-vector product still exchanges boundary values, and Krylov dot products still require collective reductions. As the local subdomain contracts, its boundary-to-interior ratio grows and communication occupies a larger share of an iteration. More ranks also increase the number of subdomains and may replicate solver or preconditioner metadata.

Weak scaling does not remove these costs, it isolates them. Holding the unknowns per core fixed keeps the local kernel work per iteration roughly constant, so any growth in the time per iteration is attributable to concurrency: more neighbours to exchange with, more ranks participating in each global reduction, and a longer reduction tree. That is why a weak-scaling curve is the appropriate place to look for the effect of replacing MPI ranks with OpenMP threads, and why Chapter 4 reports the campaign that way.

Replacing ranks with threads changes these costs rather than removing them. Larger rank-local subdomains can reduce MPI endpoints and aggregate messages, while threads work on data already inside the process. Judge’s ARCHER2 study found that two or four threads per rank often improved CP2K and CASTEP, whose runs contained expensive collective communication, but brought little benefit to the tested molecular-dynamics cases until their scaling limits [6]. The relevant conclusion is application-specific: hybrid execution helps when the saved MPI and memory costs exceed the added thread overhead.

2.3 Hybrid MPI+OpenMP

Hybrid performance depends on three coupled choices: the number of ranks, the number of threads in each rank, and their placement on the node. Increasing the thread count reduces the rank count at fixed cores, but gives each OpenMP region more threads to coordinate. Sparse linear algebra is a demanding case because its indirect memory accesses and low arithmetic intensity can saturate memory bandwidth before all cores are busy.

Lange et al. demonstrated that PETSc sparse matrix-vector multiplication could strong scale beyond a pure-MPI limit by combining MPI with task-based OpenMP, explicit communication overlap, and thread load balancing [7]. Their result establishes that hybrid PETSc execution can work, but it does not predict the performance of the current PETSc backend. Their implementation targeted a specific matrix kernel and required optimisations beyond changing the number of threads.

PETSc 3.22 added the configure option `-with-openmp-kernels` [3]. This option enables OpenMP in kernels that PETSc has annotated for threaded execution. It does not replace PETSc’s MPI decomposition or make every solver phase threaded. The build studied here uses both `-with-openmp=1` and `-with-openmp-kernels=1`. A run with one thread per rank uses the same binary as a hybrid run, which isolates the runtime layout from compiler and library differences.

This distinction is important for interpreting a negative result. Weak hybrid scaling can come from a kernel with too little work per thread, a serial or MPI-dominated phase, poor placement, or shared-memory contention. Runtime alone cannot distinguish these causes. The profiling method in Section 2.6 is needed to locate them.

2.4 ARCHER2 Node Architecture and NUMA

An ARCHER2 compute node contains two 64-core AMD EPYC 7742 (Rome) processors, giving 128 physical cores. The service configures four NUMA regions per socket, eight per node, with 16 cores and local memory channels in each region. A core has private L1 and L2 caches. Four cores form a Core Complex (CCX) and share 16 MB of L3; two CCXs form a Core Complex Die (CCD). Different CCXs do not share L3 (Figure 2.1) [8].

This topology is not taken from documentation alone. It was confirmed on an ARCHER2 compute node by a dedicated diagnostic job, because the whole interpretation of the 16×8 layout depends on where the boundaries actually fall. `lscpu` reports two sockets of 64 cores, 256 logical CPUs with simultaneous multithreading enabled in hardware, eight NUMA nodes, and 512 MiB of L3 in 32 instances, that is 16 MiB per instance. `numactl` shows each NUMA node holding exactly 16 physical cores and roughly 32 GB of memory, with relative distances of 10 within a region, 12 between regions on the same socket, and 32 across the sockets. The `sysfs` cache topology gives each L3 instance a `shared_cpu_list` of exactly four physical cores, and `hwloc-ls` shows the same 16 MB L3 above four cores. Every layout in this study is therefore placed against measured boundaries of four cores per L3 and 16 cores per NUMA region.

NUMA affects both latency and bandwidth. Linux normally allocates a page in the NUMA region of the thread that first writes it. If one thread initialises a large array and a team later accesses it from several regions, remote traffic can concentrate on the memory controllers of the first region. The ARCHER2 tuning guide also notes that two threads can approach the memory-bandwidth limit of a NUMA region for bandwidth-intensive code [9]. More threads do not guarantee more bandwidth.

Cache topology introduces a finer boundary than NUMA: four adjacent cores share an L3 cache, whereas a 16-core NUMA region contains four such L3 domains. With the close binding of Section 3.6, the four full-node layouts therefore sit differently in the hierarchy, as Table 2.2 sets out.

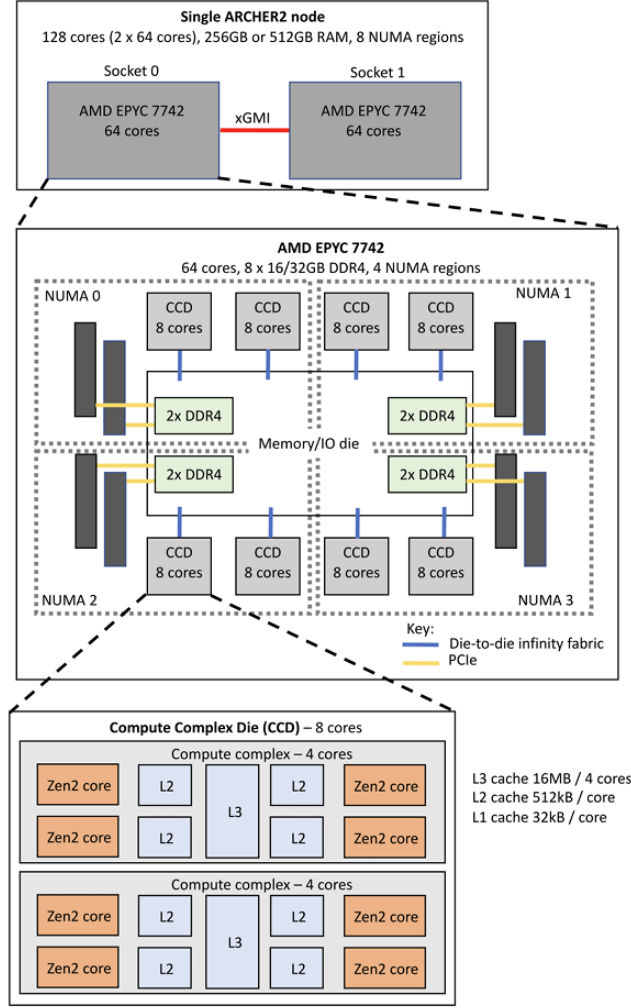


Figure 2.1: Hierarchy of an ARCHER2 compute node. Each node has two 64-core AMD EPYC 7742 processors, eight 16-core NUMA regions, and four-core CCXs that each share 16 MB of L3 cache. Source: ARCHER2 hardware documentation [8].

Table 2.2: The four fully populated layouts against the ARCHER2 node hierarchy, with close binding. Every layout uses all 128 physical cores. What changes is the number of subdomains and where each rank’s threads sit relative to the shared-L3 and NUMA boundaries. The counts are arithmetic from the node topology measured above; they describe placement, not a measured claim about cause.

Layout	Cores/rank	Subdomains/node	Rank against the shared-L3 (CCX) boundary
128×1	1	128	four ranks share one CCX
64×2	2	64	two ranks share one CCX
32×4	4	32	exactly one CCX
16×8	8	16	spans two CCXs

The last row is the one to read carefully. A rank of eight threads crosses the shared-L3 boundary, because a CCX is four cores, but it still fits inside a single 16-core NUMA region: none of the four layouts makes a rank span two NUMA regions. Any account of why 16×8 behaves differently must therefore appeal to L3 capacity or to OpenMP cost, not to NUMA placement. This mapping is a plausible mechanism for layout differences, not proof of their cause. The experiments fix thread affinity and compare measurements before using topology to interpret the ordering. Section 5.3 returns to it with profiler evidence and finds that the cost which distinguishes 16×8 is paid in the OpenMP runtime rather than at either memory boundary, which locates the cost without settling what makes the threads wait.

2.5 PETSc and Krylov Solvers

PETSc provides distributed vectors and matrices, Krylov subspace solvers (KSP), preconditioners (PC), and the logging infrastructure used in this study. PETSc uses MPI for its distributed-memory communication and lets users select solver components at run time [1, 2]. This composition is useful for experiments, but it also means that a comparison of parallel layouts is meaningful only when the numerical method performs comparable work in every layout.

The production system is the finite-difference Laplacian, whose matrix is sparse, symmetric, and positive definite. Conjugate gradients (CG) is designed for this class of linear system [10]. At each iteration CG applies the matrix and the preconditioner, updates vectors, and evaluates inner products. These operations expose two different scaling limits, and the distinction runs through the whole dissertation: the sparse matrix-vector product exchanges halo values with *neighbouring* subdomains, whereas inner products and residual norms require a reduction across *all* ranks. Table 2.3 classifies the work of one iteration accordingly.

Table 2.3: Operations in one GAMG-preconditioned CG iteration, with the PETSc event that records each and the number of calls per outer iteration measured in a representative run. Neighbour and global costs scale differently with the rank count, which is why Section 5.4 separates them.

Operation	PETSc event	Communication	Calls
Matrix-vector product	MatMult	neighbour	28.9
Multigrid restriction	MatMultTranspose	neighbour	7.0
Multigrid prolongation	MatMultAdd	neighbour	7.0
Level residual	MatResidual	neighbour	7.0
Halo exchange, all of the above	VecScatterBegin	neighbour	42.9
Inner products	VecTDot	global	2.0
Residual norm	VecNorm	global	1.0
Vector updates	VecAXPY, VecAYPX	none	44.6

The counts show why one iteration is dominated by neighbour exchange rather than by the two CG dot products: the multigrid hierarchy performs its own halo exchange on every level, so the 43 exchanges per iteration outnumber the three global reductions by more than an order of magnitude. The two nevertheless scale differently with rank count, and Section 5.4 measures both.

A preconditioner changes the spectrum of the system so that CG reaches the requested tolerance in fewer iterations. Algebraic multigrid (AMG) constructs a hierarchy directly from the matrix. Smoothing reduces high-frequency error on each level, while restriction, coarse-grid correction, and prolongation address error components that a local smoother removes slowly [11]. PETSc’s GAMG implementation supplies this hierarchy without requiring a geometric mesh

description [1].

GAMG has a setup phase and an application phase. `PCSetUp` builds the aggregates, transfer operators, and coarse matrices. Each subsequent `PCApply` traverses the multigrid hierarchy during `KSPSolve`. Applications that solve several systems with the same operator can reuse the hierarchy, so first-solve time and repeated-solve time answer different performance questions. The production scaling results in Chapter 4 and the profiling analysis both isolate repeated solves with preconditioner reuse.

The distinction also controls experimental validity. A rank-dependent block-Jacobi preconditioner changes its blocks when the rank count changes, so two rank-thread layouts would require different iteration counts for numerical rather than performance reasons. AMG has no such dependence, which is what makes a cross-layout comparison well posed. Section 3.3 gives the argument in full and the measured iteration counts.

2.6 Performance Analysis Tools

PETSc’s `-log_view` option reports time, flop counts, message counts, reductions, and event-level summaries for each logging stage [1]. It connects solver concepts to named events such as `MatMult`, `VecScatterEnd`, `VecTDot`, `PCSetUp`, and `PCApply`. The report is available for every layout, including the 128×1 MPI-only baseline, and supplies the comparable quantities used in cross-layout analysis.

Linaro MAP is a sampling profiler for MPI, OpenMP, and single-threaded programs. It combines source-level call paths with time-varying CPU, memory, vectorisation, and MPI metrics [12]. MAP can show whether ranks wait in MPI, whether OpenMP threads are imbalanced, or whether memory activity coincides with a slow phase. It also adds instrumentation and sampling overhead. This dissertation therefore uses MAP to explain behaviour within the profiled hybrid layouts and uses `-log_view` for timing comparisons across layouts. Profiled timings are not mixed with the uninstrumented scaling results.

2.7 Summary and Research Gap

Prior work gives two useful but incomplete expectations. Hybrid MPI+OpenMP often benefits communication-heavy applications on ARCHER2 at two or four threads per rank [6], and specialised OpenMP work in PETSc has extended strong scaling of sparse matrix-vector multiplication [7]. Neither result evaluates PETSc 3.24’s `-with-openmp-kernels` backend with CG+GAMG on ARCHER2, nor does either map a full rank-thread matrix across 2D and 3D stencil problems.

The unresolved question is therefore narrower than whether hybrid programming can be fast. It is whether PETSc’s current threaded kernels reduce end-to-end time for a fixed 128-core-per-node allocation, which rank-thread balance performs best as the job scales, and which solver events account for the differences. The experiments address that gap by holding total cores, solver options, and convergence criteria fixed; varying rank-thread layout, problem size, dimensionality, and node count; and linking runtime results to `-log_view` and MAP evidence.

Chapter 3

Experimental Platform and Methodology

3.1 Hardware and Software Environment

The node hierarchy relevant to placement is described in Section 2.4. All experiments used physical cores with simultaneous multithreading disabled. The software stack was PrgEnv-gnu 8.4.0, gcc 11.2.0, cray-mpich 8.1.27, and PETSc 3.24.4. The early single-node exploration used `arch-omp-opt`. The validated repeated-solve campaigns used `arch-omp-libsci-rome-opt`, configured with the threaded `libsci_gnu_mp.so.5.0`. Both builds enabled PETSc’s OpenMP kernels and used `-O3`. One thread per rank in the OpenMP-enabled build is the flat-MPI baseline.

3.2 Benchmark Problems and Drivers

Both benchmarks solve the Poisson equation $-\nabla^2 u = f$ on a rectangular domain with homogeneous Dirichlet boundary conditions, discretised by second-order central finite differences on a uniform grid. The unknowns are the interior grid points, so the boundary values never enter the linear system: a row at the edge of the grid omits the neighbours that fall outside it. Scaling by the mesh spacing is absorbed into the right-hand side, which leaves the standard stencil coefficients.

The 2D benchmark is based on PETSc’s `ex2` example. On an $m \times n$ grid it assembles the five-point stencil, diagonal 4 and off-diagonals -1 , into an $mn \times mn$ sparse matrix with five nonzeros per row. The 3D benchmark, `3d_repeat`, was written for this project because the two dimensionalities differ in surface-to-volume ratio. A subdomain of N unknowns has a boundary that grows as $N^{1/2}$ in 2D and as $N^{2/3}$ in 3D, so the two differ in how much of an iteration is spent exchanging halo values. It assembles the seven-point stencil, diagonal 6 and off-diagonals -1 , on an $m \times n \times p$ grid, with seven nonzeros per row and otherwise the same structure. Both matrices are symmetric positive definite, and both drivers set `MAT_SYMMETRIC` so that PETSc may exploit it.

Neither problem uses an analytic forcing term. Both set the exact solution to the vector of ones, $u = \mathbf{1}$, and form the right-hand side as $b = Au$. The solution is therefore known in advance, and each run reports $\|x - u\|_2$ as an end-to-end correctness check. The values quoted in Section 3.3 are that norm. This choice keeps the two dimensionalities comparable: the same construction, the same known solution, and a right-hand side that is by definition in the range of the operator. It also makes a silently wrong solve visible in every log.

The grids are chosen so that the two dimensionalities are matched in unknowns rather than in grid extent. The six sizes run from $2240^2 = 5,017,600$ to $12828^2 = 164,557,584$ unknowns in 2D and from $171^3 = 5,000,211$ to $548^3 = 164,566,592$ in 3D, so each 2D size has a 3D partner within 0.5%. Table 4.1 lists them with the node counts at which each was run.

Both production drivers, `ex2_repeat` and `3d_repeat`, use the same measurement design: one warm-up `KSPSolve` pays for the preconditioner setup, then 19 further solves run inside a dedicated `RepeatedSolves` PETSc logging stage with the operator and the preconditioner unchanged. The initial guess is zeroed before each timed solve, without which the previous solution already satisfies the tolerance and the solve returns at iteration zero. Section 5.1 gives the measurements that motivated this design.

3.3 Solver Configuration

The default `ex2` solver is GMRES with block Jacobi, one block per MPI rank, each block preconditioned by an incomplete Cholesky (ICC) factorisation. A serial run uses a single ICC factorisation of the whole matrix. At 2240×2240 (5.0M unknowns) and above, every run hit PETSc’s default limit of 10 000 iterations without converging (`DIVERGED_ITS`), with final error norms between order 10^0 and order 10^3 (up to 3.8×10^3 at 6400×6400); at 1000×1000 all rank counts except 128 converged. Those runs also did not use the tolerance their job script requested, for a reason established in Section 3.8, so the non-convergence is a joint effect of the preconditioner and of a tolerance no fixed-tolerance study would have asked for.

Production runs use

```
-ksp_type cg -pc_type gamg -ksp_rtol 1e-5 -ksp_atol 1e-10
```

CG applies because the discretised Laplacian is symmetric positive definite. GAMG is used because multigrid convergence is close to mesh-independent for elliptic problems. Across 5.0M to 164.6M unknowns, the final solve takes 9–13 iterations in 2D and 6–7 in 3D. Observed error norms range from 8.5×10^{-3} to 1.34×10^{-1} in 2D and from 6.0×10^{-3} to 8.59×10^{-2} in 3D. Small layout-dependent changes in iteration count affect close throughput comparisons, so the results report both equations solved per second and time per KSP iteration.

A second property of the default configuration matters more here than its convergence. Block Jacobi builds one block per MPI rank, so the preconditioner is a function of the rank decomposition: changing the rank–thread layout changes the operator that is applied. Table 3.1 shows this directly. Reading across any row, the iteration count does not change at all when threads are added. Reading down any column, it changes by a factor of more than three.

Table 3.1: Final iteration count of the single-node default-solver sweep on a 1000×1000 grid, for every ranks $\times$ threads ≤ 128 combination. Every row is constant: the count is fixed by the rank count alone. The 128-rank configuration reached the 10,000-iteration limit without converging.

MPI ranks	OpenMP threads per rank							
	1	2	4	8	16	32	64	128
1	6700	6700	6700	6700	6700	6700	6700	6700
2	3715	3715	3715	3715	3715	3715	3715	
4	2825	2825	2825	2825	2825	2825		
8	4085	4085	4085	4085	4085			
16	8179	8179	8179	8179				
32	6460	6460	6460					
64	7181	7181						
128	10000							

The counts are exactly reproducible: the same series appears in every repetition, and two campaigns run four months apart at 800×800 give identical iteration counts for every shared rank count. The dependence is not monotonic in the rank count, so it cannot be read as a trade-off between preconditioner strength and parallelism either. Two layouts with different rank counts

apply different preconditioners to the same linear system, and a wall-clock difference between them reflects that difference as much as any property of the layout. Cross-layout runtime under this solver is therefore indicative only, and the question “which rank–thread layout is fastest?” has no well-posed answer.

GAMG removes the direct one-block-per-rank coupling. Layouts can still differ at a fixed size and node count, but by at most two iterations in 2D and one in 3D rather than by the thousands seen with block Jacobi. That residue still prevents an unqualified claim that every layout performs identical work. Dividing by the outer KSP iteration count controls this source of variation. It does not guarantee identical GAMG hierarchy work or communication across decompositions.

GAMG cost has two phases: setup (`PCSetUp`, building the coarse hierarchy) and solve (`KSPSolve`). In a single-solve run, setup is about 45% of runtime. In applications the preconditioner is reused across solves. The production campaign and Chapter 5 both isolate the repeated solve phase. A fuller default-versus-GAMG table would require additional matched converged points and is not used for the layout claims in this dissertation.

3.4 Configuration Space and Selection Rules

Three rules reduce the space to the configurations the formal campaigns run.

Threads per rank are restricted to $\{1, 2, 4, 8\}$. Layouts that give a rank 16 or more threads cost more per KSP iteration than layouts spending the same cores on MPI ranks, and they do so on both the exploratory and the production solver configuration. Section 3.5 gives the measurements. The restriction is a search bound read off measurement, not a claim about a NUMA or cache mechanism.

Nodes are fully populated: $\text{ppn} \times \text{threads} = 128$, giving four layouts 128×1 , 64×2 , 32×4 , and 16×8 . Same node count means same core count, so a comparison isolates rank–thread balance from core count.

A configuration at size N on C physical cores is included when $25,000 \leq N/C \leq 100,000$. The completed 2D campaign uses six grids: 2240^2 , 3160^2 , 4500^2 , 6400^2 , 9073^2 , and 12828^2 , or 5.02M to 164.56M unknowns. The filter admits 11 size–node pairs over 1–32 nodes. Each admitted pair runs all four layouts and three formal repeats.

3.5 Bounding the Thread Count

The first rule of Section 3.4 is the only one that had to be measured rather than chosen, and it rests on two campaigns with complementary weaknesses. The exploratory sweep below covers every rank–thread combination on one node, but runs the default solver, whose preconditioner changes with the rank count. The production campaign that follows it runs exactly the solver, build and driver of Chapter 4 at two, four and eight nodes, but only one repeat per configuration. Both give the same answer, and the restriction is applied only because they agree.

3.5.1 Exploratory Single-Node Sweep

The sweep ran every rank–thread combination with $\text{ranks} \times \text{threads} \leq 128$ on a 1000×1000 grid (one million unknowns). It predates the solver change, so it uses the default configuration and, by the argument of Section 3.3, it cannot rank layouts with different rank counts. It can still bound the configuration space, because the conclusions drawn from it hold at a fixed rank count, where the preconditioner is also fixed. Every run reported here comes from the same build and the same campaign, and the two extremes below share one baseline: a single rank on a single thread, 245.7s at 6700 iterations.

Figure 3.1 contrasts the two ways of using the node’s 128 cores, both measured from that shared baseline. Both lines are normalised per KSP iteration, which is what makes them comparable: the threads-only line holds the preconditioner fixed by construction, while the ranks-only line changes it at every point, and its 128-rank configuration never converged.

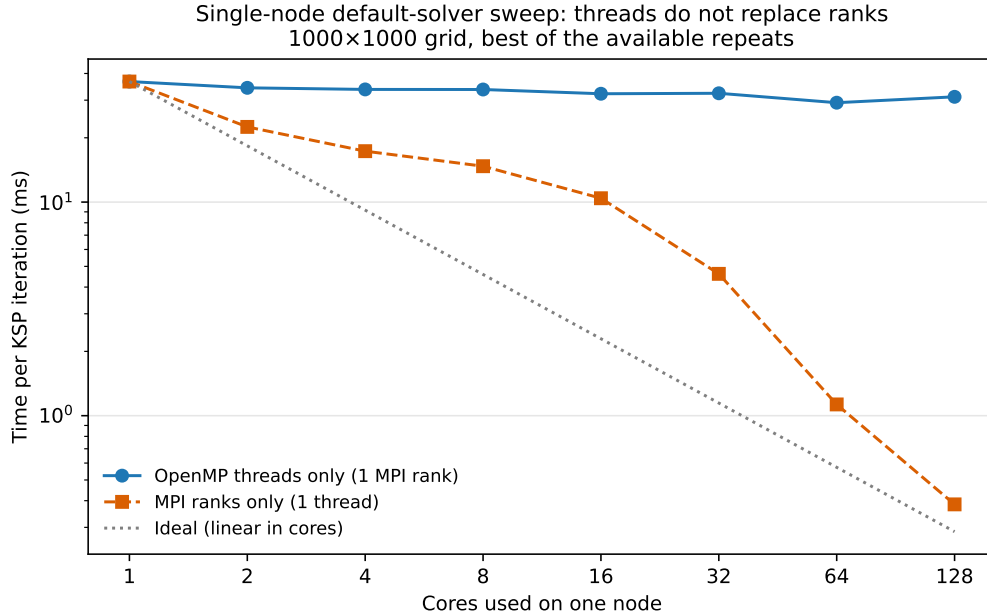


Figure 3.1: Single-node default-solver sweep on a 1000×1000 grid. Adding OpenMP threads to one MPI rank leaves the cost per iteration almost unchanged. Adding MPI ranks at one thread each reduces it by two orders of magnitude. Both lines start from the same one-rank, one-thread run.

Pure OpenMP does not scale. A single rank takes 6700 iterations at every thread count, so these runtimes are directly comparable, and they barely move: 245.7s at one thread, 225.5 at eight, 216.5 at thirty-two, 195.6 at sixty-four, and 208.4 at 128. The best case is a factor of 1.26 at 64 threads, after which more threads cost time again; 128 threads return a factor of 1.18 for 128 times the cores. Over the whole threads-only line the cost per iteration spans a factor of 1.26. Over the ranks-only line it spans 95.4. The OpenMP backend parallelises only part of the execution path, and the serial remainder bounds the achievable speedup. Chapter 5 returns to this with profiler evidence.

The MPI-only line from the same baseline reaches 8.1s on 64 ranks, a factor of 30. Its rank counts differ, so by Section 3.3 the wall-clock figure is indicative rather than a measurement of parallel efficiency. The 64-rank run performs *more* iterations than the baseline (7181 against 6700), so iteration count cannot account for any of the gap. Distributing this problem across the node works, and threading it does not.

Cost per iteration also falls monotonically as ranks replace threads at a fixed core count, as Table 3.2 shows: the trend is in the rank count, with no threshold at any particular thread count.

Table 3.2: Cost per KSP iteration for the eight fully populated single-node layouts (ranks $\times$ threads = 128) on a 1000×1000 grid. The trend is monotone in the rank count, with no threshold at any particular thread count. The 128×1 figure is measured over its non-converged 10,000 iterations, which still gives a valid per-iteration cost.

Layout	Iterations	Repeats	ms per iteration	Converged
1×128	6700	1	31.10	yes
2×64	3715	1	14.42	yes
4×32	2825	1	7.24	yes
8×16	4085	1	3.67	yes
16×8	8179	2	1.89	yes
32×4	6460	2	0.99	yes
64×2	7181	2	0.61	yes
128×1	10000	2	0.39	no

3.5.2 The Same Bound on the Production Configuration

The sweep above cannot rank layouts with different rank counts, so the restriction was re-tested on the production configuration before it was applied to the formal campaigns. A separate multi-node campaign ran the same `arch-omp-libsci-rome-opt` build, the same `ex2_repeat` driver, and the same CG+GAMG options as Chapter 4, at 2, 4 and 8 nodes, and included the two layouts this rule excludes: 8×16 and 4×32 . Table 3.3 gives the result. In all nine size-node cells the time per iteration increases monotonically with threads per rank, and the excluded layouts are not marginal: 8×16 costs 1.8–3.8 times the best layout in its cell and 4×32 costs 3.0–7.4 times. The distinction the rule draws is therefore between layouts that differ by factors, not between layouts that differ by the few per cent that separate the four admitted ones.

Table 3.3: Time per KSP iteration (ms) for the five full-node layouts of the multi-node production campaign, one run per cell, medians not available. Every layout uses all 128 cores of every node; only the rank-thread split differs. Each row is monotone from left to right, which is the property the $\{1, 2, 4, 8\}$ restriction rests on. The campaign has no 128×1 cell, and most rows sit below the 25,000 unknowns/core admission window, so this table bounds the configuration space rather than ranking the admitted layouts.

Scale	Nodes	Unknowns/core	ms per KSP iteration					Iterations
			64×2	32×4	16×8	8×16	4×32	
2m	2	7,810	1.75	2.25	3.52	6.08	11.82	9
2m	4	3,905	1.41	1.49	2.27	3.50	6.34	8–9
2m	8	1,953	1.30	1.30	1.64	2.32	3.88	8–9
4m	2	15,625	3.17	4.06	6.79	12.06	23.60	9–10
4m	4	7,813	2.13	2.52	3.80	6.46	12.39	8–9
4m	8	3,906	1.78	1.90	2.45	3.79	7.56	8–9
10m	2	39,006	11.79	13.25	20.01	30.68	61.51	9–10
10m	4	19,503	4.56	5.61	9.00	15.59	30.70	9–10
10m	8	9,752	3.10	3.28	5.05	8.65	17.03	9–10

The two smallest cells are the one caution. At 2m on eight nodes 64×2 and 32×4 differ by 0.5% (1.295 against 1.301 ms), which one run per cell cannot resolve. The ordering among the four admitted layouts is settled by the formal campaigns of Chapter 4, with three repeats and an observed range. What this campaign settles is the exclusion of 16 and 32 threads per rank on

the production solver.

Sixteen or more threads per rank is therefore excluded on both solver configurations, and the multi-node study retains 1, 2, 4 and 8.

3.6 Job Configuration and Thread Placement

Each rank and its threads are pinned to a contiguous set of cores with:

```
OMP_PLACES=cores    OMP_PROC_BIND=close
srun --hint=nomultithread --distribution=block:block --exact
```

Each part of this is load-bearing, because the whole layout comparison assumes that a layout means a fixed placement. `OMP_PLACES=cores` makes a place one physical core, so a thread cannot migrate, and `OMP_PROC_BIND=close` puts a rank’s threads on adjacent cores rather than spreading them across the node. `--hint=nomultithread` uses the 128 physical cores rather than the 256 SMT siblings. `--distribution=block:block` gives consecutive ranks consecutive cores, so ranks that exchange halos are physically adjacent. `--exact` restricts a task to its own `--cpus-per-task` cores, without which Slurm hands every task the whole node and the binding is moot. Read against the measured topology of Section 2.4, close binding on contiguous cores means a 32×4 rank occupies exactly one four-core shared-L3 domain, while a 16×8 rank occupies two. No layout places a rank across two 16-core NUMA regions. Without pinning, threads would migrate while their first-touched pages stayed put, adding remote memory traffic and run-to-run variance large enough to swamp the few-per-cent layout effect this study measures.

3.7 Metrics

The primary timing is the maximum-rank `KSPSolve` time in the `RepeatedSolves` stage. If this event has count $c = 19$, time t , and the grid has N unknowns, steady-state throughput is

$$Q = \frac{Nc}{t}.$$

The analysis also reports $t/(cI)$, where I is the final KSP iteration count. Formal repeats are aggregated by their median; whiskers show the observed minimum and maximum from three runs. Scaling is computed per layout against that layout’s smallest admitted node count. For a layout ℓ at n nodes,

$$S_\ell(n) = \frac{T_\ell(n_0)}{T_\ell(n)}, \quad E_\ell(n) = \frac{S_\ell(n)}{n/n_0},$$

where n_0 is the smallest node count run for that problem size.

3.8 Superseded Campaigns

Two earlier rounds of measurement were discarded before the campaigns of Chapter 4 were designed. Their history is given here in full rather than in a footnote, because each was undone by a defect that no plot would have revealed, each was caught by a check that is now part of the analysis pipeline, and together they account for most of the validation machinery this study carries.

The first round ran the default solver of Section 3.3, and it did not solve the problem the job script asked for. All four drivers inherit a mesh-dependent relative tolerance from upstream `ex2` before `KSPSetFromOptions` is called: $\text{rtol} = 10^{-2}/((m+1)(n+1))$ in 2D and the corresponding product in 3D (`src/ex2_repeat.c:109` and siblings). The exploratory job script overrode it with a double-dashed option, `--ksp_rtol 1e-5`, on a line whose single-dashed neighbours

demonstrably took effect. The symptom appeared first as something else entirely: iteration counts rose by 39–62% between the 800² and 1000² campaigns, which a genuine fixed tolerance of 10^{-5} cannot produce. A direct test on a compute node settles what PETSc did with the option, and is reproduced in Table 3.4.

Table 3.4: Relative tolerance actually used by `ex2` on a 100×100 grid, read from `-ksp_view`, for the three option styles. The double dash is silently ignored and the mesh-dependent default survives; `-options_left` names the unconsumed option explicitly.

Run	Option as passed	relative=	Iterations
G13-a	<code>--ksp_rtol 1e-5</code>	9.80×10^{-7}	84
G13-b	<code>-ksp_rtol 1e-5</code>	1×10^{-5}	65
G13-c	none	9.80×10^{-7}	84

The double-dashed option was never consumed: run G13-a is identical to the no-override control in both tolerance and iteration count, and PETSc’s `-options_left` report names `--ksp_rtol` as unused. The effective tolerance of those campaigns therefore tightened as $1/N$ with the mesh, reaching 2.4×10^{-10} at 6400². That is the second half of the non-convergence reported in Section 3.3: the runs were asked for a solution far more accurate than intended, with a preconditioner that also weakened as the rank count grew. The exploratory configuration differs from the production one in *both* solver and tolerance, which is why no default-versus-GAMG performance comparison is drawn anywhere in this dissertation. Section 6.2 restates the limit.

The second round changed the solver to CG with GAMG and moved to multiple nodes, covering the same problem sizes and the same four layouts as the final campaigns. It is excluded for a different reason: *the layouts it reports were never the layouts that ran*.

Every PETSc log states the number of MPI processes the library actually initialised, in the line on a `<arch>` named `<node>` with `<N>` process. Comparing that number against the rank count encoded in each filename, 141 of the campaign’s 142 two-dimensional logs disagree, and all 161 three-dimensional logs disagree. Most report a single process where 16, 32, 64 or 128 were requested. The job scripts recorded the intended layout in the filename, but the launcher did not deliver it, so a run labelled 16×8 and a run labelled 128×1 may both have executed on one MPI process. Its solve times are 20 to 50 times slower than the corresponding formal runs, which is consistent with that reading.

Any comparison between layouts drawn from those logs is therefore void, including the conclusion the earlier project material reported, that 32×4 is the fastest layout at every problem size. The apparent ordering reflects how many processes actually started, not the rank–thread balance. Two further defects would independently disqualify the campaign. It timed a single solve, so its reported `Time (sec)` is dominated by hierarchy construction: `PCSetUp` accounts for a median of 69% of it and is a median of $3.3\times$ the `KSPSolve` time. And 72 of its 88 configurations have one recorded run, so no observed range exists.

Both failures share a shape: the run produced plausible output, the filename described what was intended rather than what happened, and only the contents of the log could tell the difference. That is what the validation contract of Section 3.9 is built to catch. Every formal run in this dissertation is checked against the build, the tolerance option, and the process and thread counts PETSc itself reports. A mismatch fails the pipeline rather than producing a row; Section 4.1 reports the outcome for the campaigns this dissertation does use. The superseded logs are retained as provenance (Appendix A). The single-node sweep of Section 3.5 is a third case, unaffected by either defect: its rank counts are confirmed from the logs, its tolerance was common to every run being compared, and it is used only for conclusions that hold at a fixed rank count or per iteration.

3.9 Reproducibility

Every number, table, and figure in Chapter 4 and Chapter 5 is derived from raw job output by a script in the accompanying repository, and can be regenerated with one command per campaign. Appendix A lists the layout and the commands.

Three conventions make that possible. First, the raw `-log_view` output of every run is retained unmodified and is the only source of truth. Derived CSVs, tables, and figures are regenerated from it and are never hand-edited. Second, the analysis is content-validated rather than filename-validated. Each parser checks the build that the log itself reports, the MPI process and OpenMP thread counts PETSc actually saw, the number of solves and of timed events in the logging stage, and the linked library recorded in the embedded configure line. A row that fails any of these raises rather than being dropped. A filename is treated as a label, not as evidence. Third, the claims that the text depends on are expressed as assertions inside the scripts, so a change in the data that would invalidate a sentence in this dissertation fails the pipeline instead of passing quietly. Table 3.1 is one such case: the script that produces it refuses to write the table unless the iteration count is constant across thread counts at every fixed rank count, which is the property Section 3.3 argues from.

Thirty unit tests cover the parsers and the campaign-completeness checks and run in a single command. The measured quantities carry the limits stated where they are used: three repeats support an observed range but not a significance claim, and the exposure shares in Section 5.4 are indicators rather than an additive time budget.

The same conventions extend to the profiler evidence. Linaro MAP writes each profile as a compressed XML document, so the call-tree analysis in Section 5.3 is produced by a script that reads the `.map` files themselves, with no Forge licence and no export step in the loop. That script validates each profile’s layout against the `srun` command line recorded inside the file, rejects a profile whose frames MAP never resolved to libraries, and raises if the ordering the text argues from does not hold.

Chapter 4

2D and 3D Problems

4.1 Experimental Setup

Each dimension is covered by two campaigns. Both inherit the platform, build, drivers, solver options, thread placement and measurement contract of Chapter 3 without change, and both populate every node to all 128 physical cores. They differ in one design choice: whether the problem size grows with the node count or stays fixed. Growing it holds the work per core constant and isolates what concurrency costs. Fixing it shows how far one problem can be spread before it stops improving. Table 4.1 is the whole experiment, one row per problem size, with a column for each campaign.

Table 4.1: Grids and node counts of the six scales. The weak-scaling column is the set admitted by the unknowns-per-core filter of Section 3.4; the last column is the fixed-size campaign of Section 4.1.2, which runs the 20m problem over the whole node range and is deliberately not limited by that filter.

Scale	2D grid	2D unknowns	3D grid	3D unknowns	Weak-scaling nodes	Fixed-size nodes
5m	2240 ²	5,017,600	171 ³	5,000,211	1	—
10m	3160 ²	9,985,600	215 ³	9,938,375	1, 2	—
20m	4500 ²	20,250,000	273 ³	20,346,417	2, 4	1, 2, 4, 8, 16, 32
41m	6400 ²	40,960,000	345 ³	41,063,625	4, 8	—
82m	9073 ²	82,319,329	435 ³	82,312,875	8, 16	—
165m	12828 ²	164,557,584	548 ³	164,566,592	16, 32	—

The two node columns give 11 size–node pairs in the weak-scaling campaign and six in the fixed-size one, and every pair runs all four full-node layouts three times. The content checks of Section 3.9 were applied to each resulting log before it entered a median. All 280 logs of the weak-scaling matrix and all 144 accepted logs of the fixed-size curves pass, and none carries a divergence, PETSc/MPI/Slurm error, licence wait or `CRAYBLAS_WARNING`. The 140 retrieved 3D source-CSV rows match the independently parsed logs one for one. Eight further 20m logs per dimension are earlier pilots, retained as provenance and excluded from every median and figure.

Run-to-run spread bounds what any comparison in this chapter can resolve. Over three repeats the largest min–max spread is 4.71% in 2D and 3.83% in 3D for the weak-scaling matrix, and 10.77% and 11.09% for the fixed-size curves, which are noisier because a solve at their largest node counts takes only a few tens of milliseconds. A layout gap below those figures is reported as a tie rather than a win.

4.1.1 Weak-Scaling Chains

The admission filter $25,000 \leq N/C \leq 100,000$ couples problem size to node count: each scale survives at only one or two node counts, so a panel drawn per problem size contains two points. The same 11 size–node pairs, however, decompose exactly into two chains along which the unknowns per core stay near-constant (Table 4.2).

Table 4.2: The 11 admitted size–node pairs, ordered into two weak-scaling chains. Each row holds unknowns per core near-constant while the node count doubles, so a layout’s cost along a row isolates the effect of concurrency at fixed local work.

Band	Size at node count					
	1	2	4	8	16	32
~40k unknowns/core	5m	10m	20m	41m	82m	165m
~80k unknowns/core	10m	20m	41m	82m	165m	—

The chains are six and five points long, and every configuration on them falls within 15% of its nominal band. This dissertation therefore reports the campaign as weak scaling along these two chains, and treats the per-size node doublings as a separate, local observation. Ideal weak scaling would hold the time per iteration constant along a row. The measured growth is the cost of concurrency at fixed local work.

4.1.2 Strong-Scaling Design

Five sizes in each dimension have measurements at two adjacent node counts. These are strong-scaling steps, orthogonal to the weak-scaling chains above, and each spans one doubling only, which cannot show where a layout stops scaling. A matched extension therefore holds the problem size fixed and runs the full node sequence $\{1, 2, 4, 8, 16, 32\}$, giving six-point curves for every layout in both dimensions; nothing else changes. The fixed problems are the 20m row of Table 4.1, and at 32 nodes they leave about 4.9 thousand unknowns per core, deliberately past the admission window of Section 3.4. That point is retained as the saturation tail and should not be read as an efficient operating point.

Two consistency checks were run on this campaign, and neither changes anything reported later. Its 2- and 4-node points repeat size–node pairs that the weak-scaling matrix of Section 4.2 collected in an earlier session. In 2D the medians differ by at most 2.2%, with six of the eight observed ranges overlapping. In 3D the reruns are 0.3–3.1% slower and no range overlaps, so the two campaigns are reported separately rather than pooled. The 3D campaign’s 1-, 2- and 32-node points were also submitted twice. The analysis uses the newer complete batch of three repeats and keeps the earlier 36 logs as superseded provenance. Those two batches measure reproducibility across job submissions rather than within one job. Their medians agree to within 0.78% at one and two nodes and differ by -4.7% to $+5.1\%$ at 32 nodes, where a solve takes only tens of milliseconds. In both batches 16×8 is the fastest layout at 32 nodes, ahead of flat MPI by $3.81\times$ and $4.00\times$, so Section 4.3.2 does not depend on which batch is used.

4.2 Weak Scaling at Constant Work per Core

Both dimensions run the same 11 size–node pairs, the same four layouts and the same three repeats, so the two subsections below describe each on identical axes before Section 4.2.3 ranks the layouts across both.

4.2.1 2D

Raw throughput rises almost linearly with the node count along both chains, and the four layouts sit within a few per cent of one another against a three-order-of-magnitude range (Figure B.1). That axis therefore separates the layouts only weakly: the dominant signal is the node count, while the comparison that matters is between layouts at a fixed point. Figure 4.1 divides each layout’s time per KSP iteration by that of flat MPI at the same size–node pair. This normalisation removes both the scale and the 9–13 GAMG iterations, leaving only the layout effect.

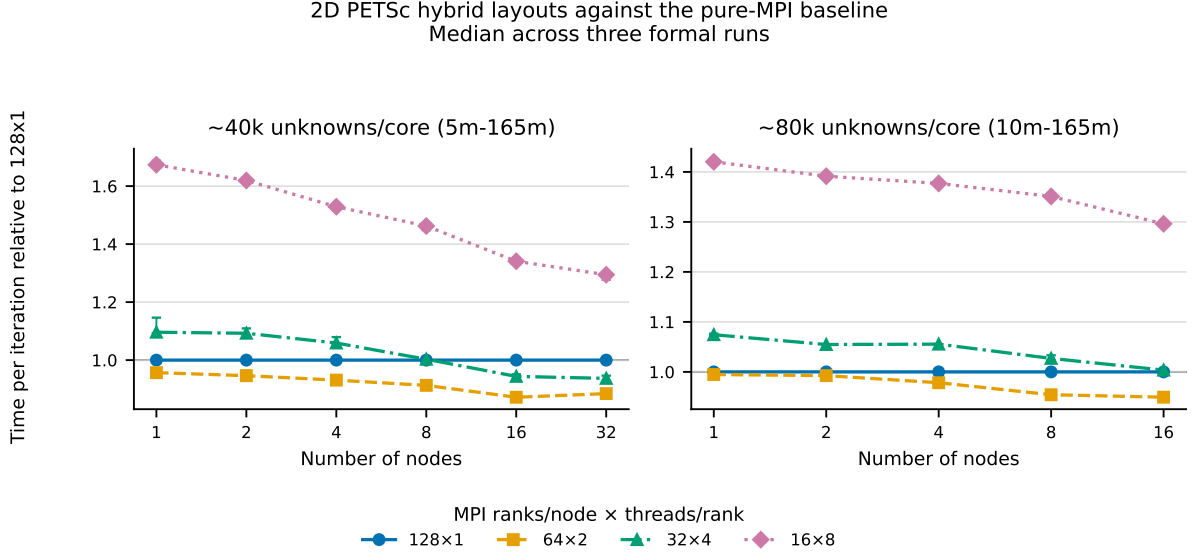


Figure 4.1: 2D time per KSP iteration relative to the 128×1 baseline at the same size–node pair; values below 1 mean the hybrid layout is faster. Whiskers propagate the observed range of three formal runs.

The result is an ordered trend rather than a set of isolated wins. Along the $\sim 40k$ chain, 64×2 falls from 0.957 at one node to 0.872 at 16 nodes, 32×4 from 1.096 to 0.937, and 16×8 from 1.674 to 1.294. Along the $\sim 80k$ chain all three ratios decrease at every step: 64×2 from 0.995 to 0.949, 32×4 from 1.074 to 1.004, and 16×8 from 1.420 to 1.296. Four of the six series are strictly monotone, and the two exceptions each contain a single small reversal (64×2 from 0.872 to 0.885 at the last $\sim 40k$ point, and one flat step in 32×4).

Two consequences follow. First, 64×2 is faster than flat MPI per iteration at every one of the 11 pairs, and its margin widens with concurrency rather than fluctuating. Second, 32×4 is not a fixed loser but a crossing curve: it costs 9.6% more per iteration than flat MPI at one node, reaches parity at eight nodes (1.003), and is 6.3% cheaper at 32 nodes. A layout ranking measured at small node counts does not transfer to large ones.

Raw throughput adds the convergence effect back in and is therefore noisier: 64×2 has the highest median at eight of the 11 pairs and flat MPI at the remaining three. Two of those three flat-MPI wins coincide with flat MPI needing one fewer GAMG iteration. At the third, 41m on eight nodes, the two medians differ by 0.4% and their observed ranges overlap, which is a tie rather than a win. At 164.56 million unknowns on 32 nodes, 64×2 reaches 794.8 million equations per second, 13.1% above flat MPI. Its largest throughput margin over the matrix is 14.7%, at 82m on 16 nodes, which is the same point as its largest per-iteration saving of 12.8%.

16×8 has the lowest throughput at all 11 pairs and never approaches the baseline, but the size of its deficit is not constant: it narrows from 46.2% at 5m on one node to 22.7% at 165m on 32 nodes.

Figure 4.2 gives the absolute time per KSP iteration along each chain. Ideal weak scaling

would be a horizontal line.

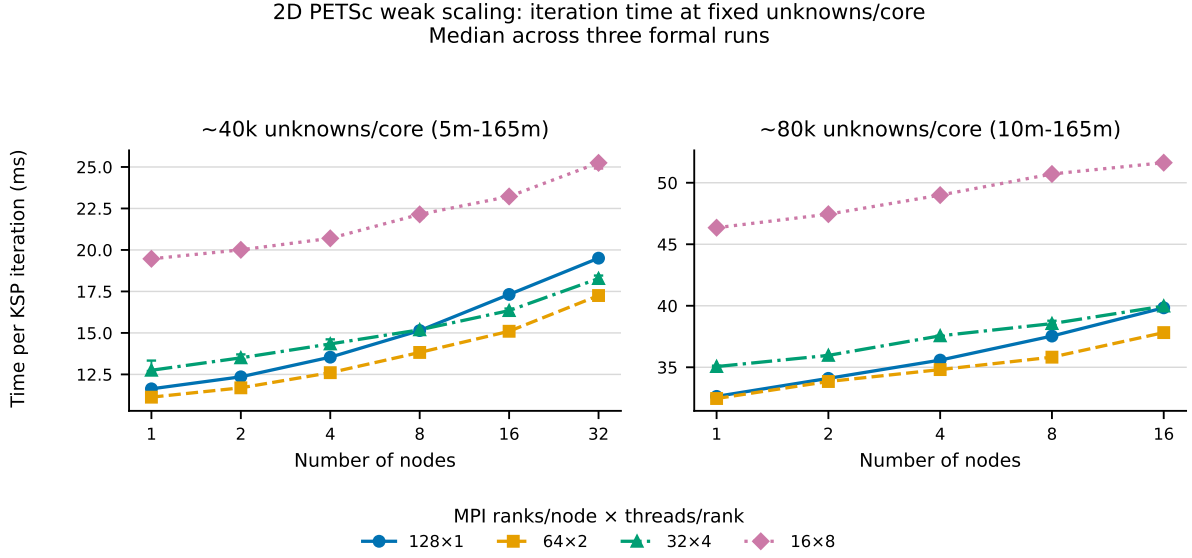


Figure 4.2: Median 2D time per KSP iteration along the two unknowns/core chains. A horizontal line would be ideal weak scaling.

4.2.2 3D

The corrected 3D campaign uses the same size–node points, layouts, repeat count, and measurement contract. The final KSP iteration count ranges from six to seven. Its raw throughput has the same near-linear shape and the same weak layout separation as the 2D case (Figure B.2), so the comparison is again made on the normalised ratio of Figure 4.3.

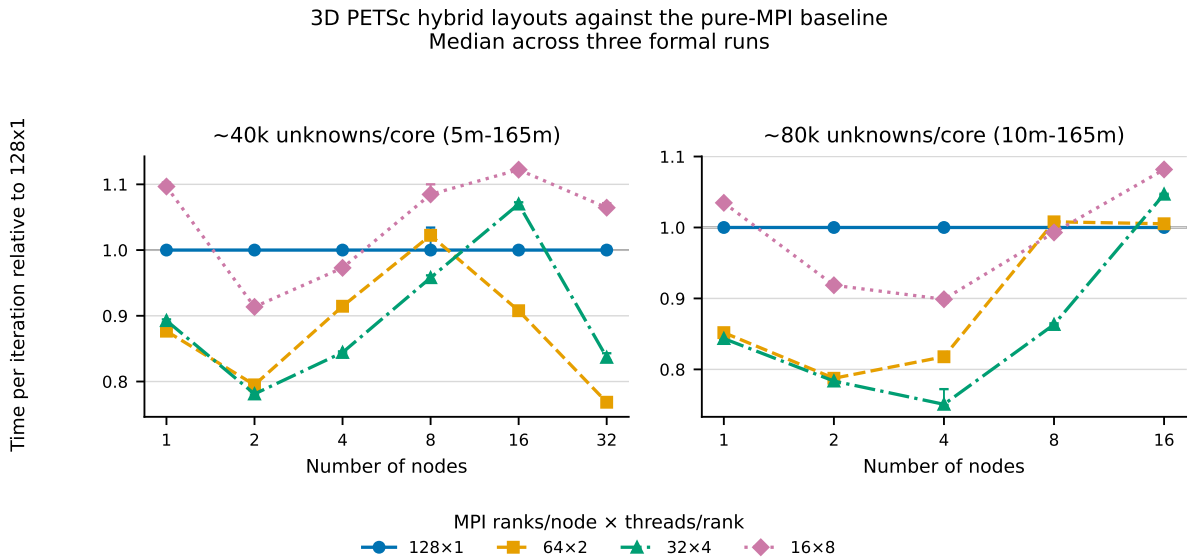


Figure 4.3: 3D time per KSP iteration relative to the 128×1 baseline at the same size–node pair. Unlike the 2D case in Figure 4.1, no series is monotone.

The 3D behaviour differs from 2D in both magnitude and shape. The advantage is larger: the best measured point is 32×4 at 41m on four nodes, at a ratio of 0.751, so it takes 24.9%

less time per iteration than flat MPI where the 2D best takes 12.8% less. The same point is the largest raw-throughput margin in either campaign, 33.2% against a 2D best of 14.7%. The two pairs of figures differ because a throughput margin is a rate ratio and a time saving is not. Two-thirds of the 33 hybrid measurements fall below 1, against 13 of 33 in 2D.

The advantage is also not ordered by concurrency. Every one of the six series is non-monotone. Along the $\sim 80k$ chain the hybrid layouts are strongest at two and four nodes (32×4 reaching 0.784 and 0.751), lose the advantage by eight nodes, and are at or above parity at 16 nodes, where flat MPI has the lowest time per iteration of any layout. Along the $\sim 40k$ chain the pattern recovers at the largest point: 64×2 returns to 0.768 at 165m on 32 nodes after sitting at 1.022 at 41m on eight nodes.

Raw throughput selects 32×4 at six of the 11 pairs, 64×2 at four, and flat MPI at one. Iteration-normalised time selects 32×4 at seven, 64×2 at three, and flat MPI at one. At 165m on 32 nodes, 64×2 reaches 510.4 million equations per second, 30.1% above flat MPI. 16×8 wins no point, but unlike the 2D case it is not uniformly the slowest layout: it is faster than flat MPI at five of the 11 pairs.

Figure 4.4 gives the absolute time per KSP iteration along each chain, on the same convention as the 2D case.

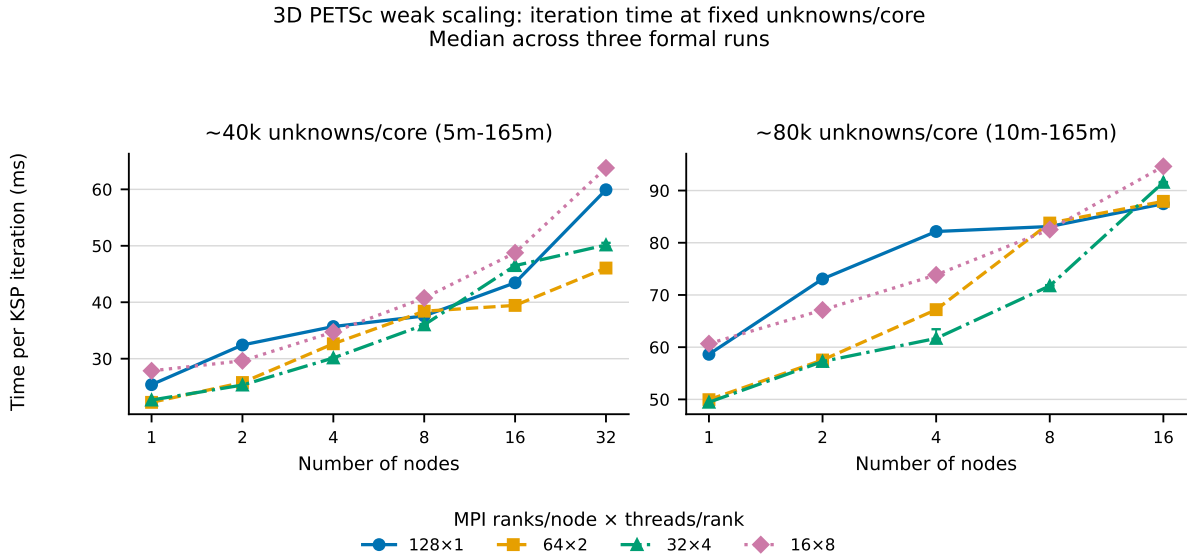


Figure 4.4: Median 3D time per KSP iteration along the two unknowns/core chains.

4.2.3 Layout Ranking and Efficiency

Table 4.3 certifies the absolute values behind both sets of ratios. For every admitted size–node pair it gives the fastest layout, what it achieved, and its margin over flat MPI on the same cores; it is the weak-scaling counterpart of Table 4.5.

The two blocks differ in stability as much as in which layout wins. Reading down the 11 rows, the 2D winner changes twice and the 3D winner changes five times. That is the same contrast the ratio figures carry: in 2D an ordered trend with one layout ahead throughout, in 3D a preference that moves with both scale and node count.

Taking each layout’s own single-node time as its baseline gives the weak-scaling efficiencies in Table 4.4.

In 2D the ordering is strict and identical in both chains: efficiency increases with the number of threads per rank, and 16×8 degrades least of all four layouts. This is consistent with the trend in Figure 4.1: replacing MPI ranks with threads reduces the part of the iteration that

Table 4.3: Fastest full-node layout at every admitted size–node pair, by median raw throughput over three formal repeats, with its margin over flat MPI at the same core count. A margin of 0.0% means flat MPI itself was fastest. The 2D 41m/8-node row is a tie rather than a win: the two medians differ by 0.4% and their observed ranges overlap. Throughput folds in the GAMG iteration count, which is why the last column is given.

Scale	Nodes	Cores	Best layout	M eq/s	vs. MPI	ms/iteration
<i>2D, five-point stencil</i>						
5m	1	128	128×1	47.9	0.0%	11.63
10m	1	128	128×1	34.0	0.0%	32.63
10m	2	256	64×2	94.9	5.7%	11.69
20m	2	256	64×2	59.8	0.8%	33.84
20m	4	512	64×2	160.7	7.4%	12.60
41m	4	512	64×2	107.0	2.2%	34.81
41m	8	1024	128×1	270.5	0.0%	15.14
82m	8	1024	64×2	208.9	4.8%	35.82
82m	16	2048	64×2	495.6	14.7%	15.10
165m	16	2048	64×2	362.6	5.3%	37.82
165m	32	4096	64×2	794.8	13.1%	17.25
<i>3D, seven-point stencil</i>						
5m	1	128	64×2	37.4	14.1%	22.26
10m	1	128	32×4	28.7	1.6%	49.45
10m	2	256	64×2	64.3	25.8%	25.78
20m	2	256	32×4	50.8	27.6%	57.26
20m	4	512	32×4	96.5	1.5%	30.13
41m	4	512	32×4	95.1	33.2%	61.68
41m	8	1024	32×4	163.0	4.4%	35.99
82m	8	1024	32×4	163.9	15.8%	71.75
82m	16	2048	64×2	298.2	10.2%	39.44
165m	16	2048	128×1	268.8	0.0%	87.47
165m	32	4096	64×2	510.4	30.1%	46.06

Table 4.4: Weak-scaling efficiency (%) at the largest node count of each chain, relative to the same layout’s one-node time. Each layout is measured against itself, so this quantifies degradation, not absolute speed.

Case	Chain and node count	Layout			
		128×1	64×2	32×4	16×8
2D	~40k, 32 nodes	59.6	64.5	69.8	77.1
2D	~80k, 16 nodes	81.9	85.8	87.7	89.8
3D	~40k, 32 nodes	42.4	48.3	45.2	43.7
3D	~80k, 16 nodes	67.0	56.8	54.0	64.1

grows with node count, which is why the ratio curves fall. The two metrics measure different things and must not be conflated. 16×8 has the best weak-scaling efficiency and the worst absolute time at every point. It degrades least because its per-iteration cost is already dominated by a term that does not grow with the node count. Weak-scaling efficiency measured against a layout’s own baseline rewards a poor baseline.

The 3D rows of Table 4.4 show no ordering by thread count. 64×2 degrades least on the $\sim 40k$ chain, while on the $\sim 80k$ chain flat MPI degrades least and both intermediate hybrid layouts degrade more than either extreme. The 2D mechanism, whatever it is, does not carry over.

4.3 Strong Scaling at Fixed Problem Size

The fixed-size campaign of Section 4.1.2 replaces constant work per core with a constant problem, so the question changes from what concurrency costs to where it stops paying. The same six-point node sequence and the same four layouts are used in both dimensions.

4.3.1 2D

Across the five per-size node doublings the four 2D layouts give speedups from 2.00 to 3.08, or apparent efficiencies from 100.1% to 154.2%. The superlinear values are repeatable within this three-run sample, but the experiment does not identify their cause. Cache capacity, changes in GAMG’s distributed hierarchy, and system-level variation all remain possible.

The fixed-size curves of Section 4.1.2 extend this from one doubling to five. The 2D curves in Figure 4.5 scale well and stay together. Every layout is still improving at 32 nodes, and the spread between fastest and slowest widens only from $1.47\times$ at one node to $1.81\times$ at 16 before narrowing again to $1.48\times$ at 32. 16×8 starts 47% slower than flat MPI and ends 45% slower, while 64×2 and flat MPI end within 2% of each other. The one-node ordering is also almost the 32-node ordering, differing only by 64×2 and flat MPI swapping places: 64×2 is fastest at nodes $\{2, 4, 8, 32\}$ and flat MPI at $\{1, 16\}$. Cumulative efficiency exceeds 100% between two and 16 nodes, peaking at 191% for 64×2 at eight nodes. As with the single doublings above, these superlinear values are repeatable but unexplained by this experiment; the fixed problem shrinking into cache is a candidate it cannot test.

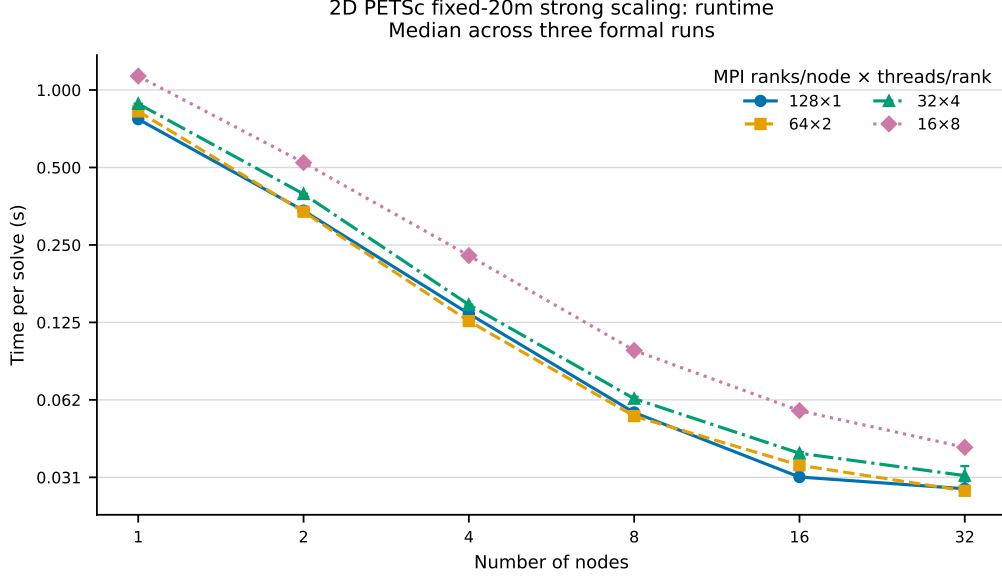


Figure 4.5: 2D time per solve at a fixed 20.25M unknowns, one to 32 nodes. Both axes are logarithmic, so ideal strong scaling is a straight line of slope -1 . Whiskers show the observed range of three formal runs.

4.3.2 3D

Across the five per-size node doublings the 3D speedups range from 1.46 to 2.39, with no superlinear step. At 165m the 16-to-32-node step gives $1.91\times$ for 64×2 against $1.46\times$ for flat MPI.

The fixed-size result in Figure 4.6 is qualitatively different from its 2D counterpart, and it is the strongest layout effect measured anywhere in this study. Every 3D layout eventually stops scaling and gets *slower*, but they do so at different node counts, and the order is the thread count. Flat MPI turns first, reaching its minimum at eight nodes, 0.0997 s per solve, then 0.1776 s at 16 and 0.2599 s at 32. 64×2 and 32×4 turn one doubling later, both bottoming at 16 nodes. 16×8 has not turned by 32 nodes, where it is the fastest of the four at 0.0681 s per solve, $3.81\times$ faster than flat MPI on the same 4096 cores and 10% faster than 32×4 , which has already begun to rise. The observed ranges at that point are disjoint by a wide margin. That the saturation node count increases monotonically with threads per rank, 8, 16, 16, and beyond 32, is what hybridisation buys in 3D.

Figure 4.7 restates the 3D case as parallel efficiency, which is the form a reader is most likely to have seen for other codes. Each layout is measured against its own one-node time, so the curves answer "how well does this layout scale?" and not "which layout is fastest?". The two questions have different answers here, and Section 4.3.3 gives both. No 2D counterpart is drawn: the 2D curves do not turn over, and their efficiencies are read directly from Table 4.6.

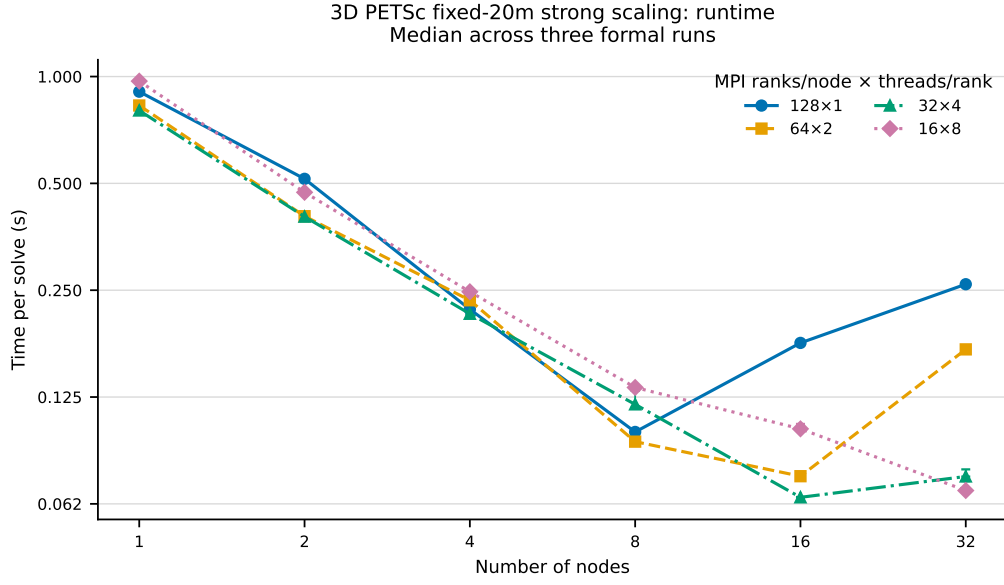


Figure 4.6: 3D time per solve at a fixed 20.35M unknowns, one to 32 nodes. Flat MPI and 64×2 reach a minimum and then get slower, while the two most-threaded layouts continue to improve.

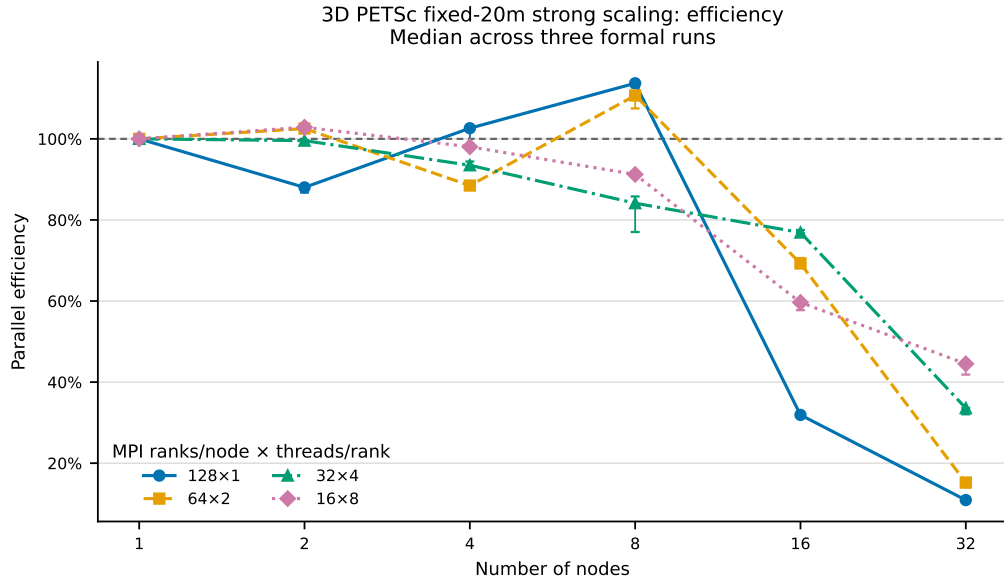


Figure 4.7: 3D parallel efficiency at a fixed 20.35M unknowns, each layout against its own one-node time. Because the baselines differ, this figure ranks scaling behaviour and not speed; flat MPI falls to 10.9% at 32 nodes while 16×8 retains 44.5%.

4.3.3 Endpoints

Table 4.5 gives the fastest layout at every node count in both dimensions and its margin over flat MPI. In 2D that margin never exceeds 6.9%. The 3D columns restate the saturation result of Section 4.3.2 as a margin instead of a turning point: 172.4% at 16 nodes and 281.3% at 32.

Table 4.5: Fastest full-node layout at each node count for the fixed 20M problems, with its median time per solve and its advantage over flat MPI at the same core count. Medians of three runs; the 32-node row has about 4.9 thousand unknowns per core.

Nodes	Cores	2D, 4500 ²			3D, 273 ³		
		Best	s/solve	vs. MPI	Best	s/solve	vs. MPI
1	128	128×1	0.7704	0.0%	32×4	0.8022	13.0%
2	256	64×2	0.3365	1.0%	32×4	0.4028	27.8%
4	512	64×2	0.1265	6.9%	32×4	0.2145	2.9%
8	1024	64×2	0.0541	3.3%	64×2	0.0936	6.5%
16	2048	128×1	0.0313	0.0%	32×4	0.0652	172.4%
32	4096	64×2	0.0277	1.9%	16×8	0.0681	281.3%

Table 4.6 gives all four layouts at the 32-node endpoint in both dimensions. It also shows why cumulative efficiency alone would mislead here. In 2D the four layouts reach almost the same speedup, 27.3 to 29.8, because each is measured against its own one-node time and 16×8 starts from the worst baseline. The runtimes they reach still differ by 45%. In 3D the two measures agree, because the layouts diverge rather than converge.

Table 4.6: The 32-node endpoint of the fixed-size curves. Speedup and cumulative efficiency are measured against the same layout’s one-node time, so they reward a poor baseline; the doubling efficiency is the 16-to-32-node step alone. Time per solve is the only column that compares layouts directly.

Case	Layout	s/solve	Speedup	Efficiency	16→32 step
2D	128×1	0.0282	27.30	85.3%	55.5%
2D	64×2	0.0277	29.83	93.2%	62.9%
2D	32×4	0.0317	27.85	87.0%	61.1%
2D	16×8	0.0409	27.71	86.6%	69.4%
3D	128×1	0.2599	3.49	10.9%	34.2%
3D	64×2	0.1703	4.87	15.2%	22.0%
3D	32×4	0.0747	10.75	33.6%	43.7%
3D	16×8	0.0681	14.24	44.5%	74.6%

4.4 Interpretation

The two dimensions differ in the shape of the result, not only in which layout wins. Table 4.7 collects the comparison in one place. Its two halves carry different weight: the 3D gaps against flat MPI at 16 and 32 nodes, $2.72\times$ and $3.81\times$, are outside the run-to-run spreads of Section 4.1, whereas the 1–7% margins the 2D layouts hold on the fixed problem are not, so the 2D ordering at any single node count is indicative rather than decisive.

The pattern across the table is that 2D is ordered and 3D is not. Every 2D hybrid layout improves relative to flat MPI as concurrency grows, monotonically along both chains, and weak-scaling efficiency rises with threads per rank. In 3D the margins are roughly twice as

Table 4.7: The two dimensions on the quantities this chapter compares them by. The upper block is the weak-scaling matrix at constant work per core, the lower block the fixed-size curves of Section 4.1.2. Throughput gain and time saving are different conventions and are both given, because the two dimensions are quoted in both.

	2D	3D
<i>Constant work per core, 11 size–node pairs</i>		
Layout winning most points	64×2 (8 of 11)	32×4 (6 of 11)
Hybrid points faster per iteration	13 of 33	22 of 33
Largest throughput gain	14.7%	33.2%
Largest per-iteration saving	12.8%	24.9%
Ratio series monotone in nodes	4 of 6	0 of 6
Weak efficiency rises with threads	yes	no
<i>Fixed 20M problem, one to 32 nodes</i>		
Layouts that saturate by 32 nodes	none of 4	3 of 4
First to saturate	—	128×1 , at 8 nodes
Fastest at 32 nodes	64×2	16×8
Its lead over flat MPI	$1.02\times$	$3.81\times$
Fastest-to-slowest spread there	$1.48\times$	$3.81\times$

large, yet no series is monotone, the preferred layout changes with scale and node count, and flat MPI degrades least on one of the two chains. The regimes differ as much as the dimensions: at constant work per core the layout effect is a matter of per cent, while at constant problem size it is unbounded, because the layouts differ in where they stop scaling rather than only in speed.

The five-point and seven-point stencils perform different work per equation, so their absolute equations-per-second rates are not direct measures of relative algorithm speed. The cross-dimensional evidence concerns layout ordering, margins relative to flat MPI, and the shape of each trend.

Two results in this chapter are not explained by it. The first is the ordering. Replacing MPI ranks with threads improves matters as concurrency grows in 2D, and the weak-scaling efficiencies of Table 4.4 order exactly by thread count, which is what a communication-share account would predict. This chapter measures runtime, not communication, and cannot tell that account from any other. The second is the counter-example. 16×8 removes the most MPI endpoints of any layout and is nevertheless the slowest 2D layout at every one of the 11 points, so whatever hybridisation saves, it also buys a cost that grows with the thread count. Chapter 5 measures both quantities directly, on these same runs, and Chapter 6 turns the outcome into recommendations.

What this chapter does settle on its own is that a layout recommendation is incomplete without the scaling regime it was measured in. The same 16×8 layout is last at constant work per core in both dimensions and first at 4096 cores on a fixed 3D problem, by a factor of 3.81 over flat MPI. Neither reading is wrong. They answer different questions, and only the second one is about where a fixed problem stops scaling.

Chapter 5

Profiling and Bottleneck Analysis

5.1 Why a Repeated-Solve Driver

In a single solve at 20.25M with 16×8 , `-log_view` gave `PCSetUp` 45% of time and `KSPSolve` 10%. `ex2_repeat` runs the solve inside a `RepeatedSolves` log stage, zeroes the initial guess before each solve, and reuses the preconditioner (`-ksp_reuse_preconditioner` and `-pc_gamg_reuse_interpolation true`). With 20 solves the stage held 69% of time and 93% of flops; within the stage `KSPSolve` was 100% of time; each of the 19 repeats took 11 iterations. The corrected scaling campaign uses this same warm-up and reuse design, so Chapter 4 compares steady-state solves rather than first-touch and hierarchy-construction costs.

5.2 Analysis Methodology

The method combines MAP 25.0.4 timelines with `-log_view %T` (share of stage time) and `%F` (share of stage flops). An event whose `%T` exceeds its `%F` spends the difference on communication, synchronisation, or memory traffic. Table 5.1 lists single-point data for the `RepeatedSolves` stage at 20.25M with 16×8 .

Table 5.1: Per-event time and flop shares, `RepeatedSolves` stage, 20.25M, 16×8 .

Event	%T	%F	GF/s
<code>MatMult</code>	39	55	30.5
<code>MatMultTranspose</code>	18	6	6.7
<code>VecTDot+VecNorm</code>	8	6	—
<code>PCApply</code> (agg.)	82	82	21.6

`MatMultTranspose` (GAMG restriction) has a `%T` three times its `%F` and a rate $4.6\times$ below `MatMult`. `VecTDot` and `VecNorm` are the CG global reductions.

5.3 MAP Profiling Matrix Results

Two MAP campaigns profiled the same 20.25M 2D problem: a baseline set on `arch-omp-opt` at 2, 8, and 16 nodes, and a matching set on the production `arch-omp-libsci-rome-opt` build at 2 and 8 nodes. Neither contains a 128×1 cell, because MAP cannot launch 128 MPI ranks per node on this system. Flat MPI therefore has `-log_view` evidence only. Of the twelve profiles, eleven are used here. The 16-node 16×8 baseline profile is rejected on its own content: MAP resolved no library symbols in it, so every sample is attributed to its own sampler and the file carries no cost information.

A `.map` file is a compressed XML document whose `sample` elements are whole-job snapshots of the merged call tree, each node carrying the number of cores active in that call path. Summing the active cores over the leaves of the tree partitions the job’s core-time by the code that was actually running, which is how MAP itself forms its percentages, and it can be done from the file alone. The analysis here does that, validating each profile’s layout against the `srun` line recorded inside the file rather than against its name, and reports two scopes. The whole-program scope covers every sample. The solve-window scope keeps only samples in which cores are inside `KSPSolve` and none is inside `PCSetUp`, which approximates the repeated-solve phase from the profile alone. It is a time window, not the PETSc `RepeatedSolves` logging stage, and the two must not be quoted as the same quantity.

Table 5.2 gives the resulting solve-window shares for all eleven profiles, and Figure 5.1 plots them.

Table 5.2: Share of total core-time inside the solve window, by the library owning the executing frame, for the eleven usable MAP profiles at 20.25M unknowns in 2D. OpenMP is the GNU runtime’s spin and wait paths, that is threads with no work. MAP instrumentation inflates these runs, so the ordering between layouts is the result. The absolute values are not transferable to the uninstrumented campaigns.

Build	Nodes	Layout	PETSc	OpenMP	MPI	LibSci
LibSci	2	64×2	31.9%	36.5%	29.9%	1.0%
LibSci	2	32×4	51.6%	40.1%	5.8%	1.7%
LibSci	2	16×8	42.2%	53.5%	1.4%	2.1%
LibSci	8	64×2	4.5%	48.3%	46.7%	0.2%
LibSci	8	32×4	7.5%	69.3%	22.6%	0.3%
LibSci	8	16×8	15.6%	76.5%	6.2%	1.0%
Baseline	2	32×4	51.7%	40.8%	6.7%	0.0%
Baseline	2	16×8	40.2%	57.8%	1.4%	0.0%
Baseline	8	32×4	8.8%	68.6%	22.2%	0.0%
Baseline	8	16×8	16.0%	77.5%	6.0%	0.0%
Baseline	16	32×4	1.2%	69.8%	29.0%	0.0%

The two costs trade against each other. The MPI share falls as threads replace ranks: at eight nodes it goes 46.7%, 22.6% and 6.2% across 64×2 , 32×4 and 16×8 . That is the ordering Section 5.4 obtains from `-log_view` on the whole formal matrix, reproduced here by a different instrument on a different quantity. The time spent in the OpenMP runtime runs the other way, 48.3%, 69.3%, 76.5%, and it is not useful work: the frames are `cpu_relax`, `do_spin` and `__futex_wait`, that is threads waiting for something to do. The two builds agree wherever they overlap: across the four common cells the MPI shares differ by at most 0.9 percentage points and the OpenMP shares by at most 4.3, with no systematic sign. This is a property of the layout, not of the linked library.

Three limits apply. MAP’s overhead is large and uneven (separately measured at +35% on the timed stage and roughly threefold on `VecScatterEnd`), and its OpenMP instrumentation perturbs exactly the barrier behaviour being measured, so the absolute percentages overstate what an uninstrumented run would show. There is one profile per cell, so there is no observed range. And with no 128×1 profile the trend is anchored only at its hybrid end, though flat MPI has no OpenMP runtime time at all by construction, which fixes the missing point at zero.

Within those limits the table answers the question Chapter 4 left open, at the level of accounting rather than of mechanism. The cost that grows with threads per rank, and that must be repaid before hybridisation helps, is paid as idle time in the OpenMP runtime: the term that grows is time in `cpu_relax`, `do_spin` and `__futex_wait`, not time in the compute kernels.

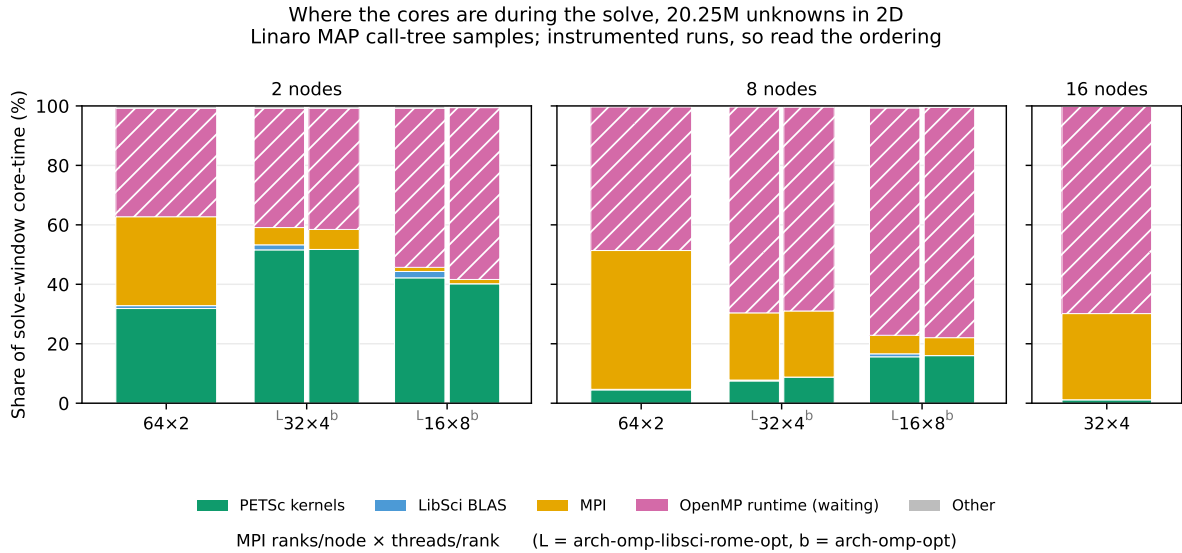


Figure 5.1: Where the cores are during the solve, from the MAP call trees. Within each panel the OpenMP waiting share grows and the MPI share shrinks as threads replace ranks, and both effects strengthen with node count. Paired bars are the two PETSc builds at the same configuration.

NUMA placement is excluded on topology alone: a rank of eight threads crosses a shared-L3 boundary but still fits inside one 16-core NUMA region, so no layout of this study places a rank across two NUMA regions and the four cannot differ in that respect (Table 2.2).

Neither is the growth a placement effect, because it is present at fixed layout, where a rank’s threads occupy the same cores against the same shared-L3 boundary at every node count: holding 16×8 and going from two nodes to eight raises the waiting share from 53.5% to 76.5% on the LibSci build and from 57.8% to 77.5% on the baseline, as the work per threaded region shrinks.

That is the discriminating observation. The other two features of the shape, monotonicity in the thread count and a maximum exactly where 16×8 performs worst, are consistent with this reading but equally consistent with a cache one. What the profiles do not settle is why the threads wait: a rank spanning two CCXs could produce imbalance that is collected at the same barrier, and Section 5.4.4 leaves that to the hardware-counter work of Section 7.2. Section 5.4 shows the other half of the trade, on the full matrix and without a profiler.

A fourth observation is negative and worth recording. Threaded LibSci is linked into the production build and PETSc’s sparse solve path barely calls it: BLAS accounts for at most 2.1% of solve-window core-time in any LibSci profile, and the baseline profiles sample no BLAS frame at all. The threaded BLAS library therefore cannot account for the layout ordering measured in Chapter 4, which confirms from profile data what Section 6.2 could previously only state as a limit of the evidence.

5.4 Bottleneck Identification

Section 5.3 leaves the hardware question open, but the `-log_view` event tables of the uninstrumented formal runs support a narrower question that Chapter 4 raised and could not answer: whether the layout effect tracks the share of the solve exposed to inter-process synchronisation. All 264 formal runs are parsed here, so the evidence covers the same matrix as the scaling results and carries no profiler overhead.

5.4.1 What is Measured

For each run, the time of the halo-exchange events (`VecScatterBegin`, `VecScatterEnd`) and of CG’s global-reduction events (`VecTDot`, `VecNorm`) is taken from the `RepeatedSolves` stage and divided by that stage’s `KSPSolve` time. Three limits apply and none is removable from `-log_view` alone. The synchronising events are logged *inside* `MatMult` and its relatives, so this is an exposure share and not a term in an additive time budget. Summing every leaf event against `KSPSolve` gives 116–372%. Each event time is a maximum over ranks, and different events may attain their maximum on different ranks. `VecTDot` and `VecNorm` also perform local flops, so their time is not purely communication. The quantity is nevertheless measured identically in every run, which is what makes it comparable across layouts and node counts.

5.4.2 Halo Exchange Dominates, and Hybridisation Reduces It

Figure 5.2 and Figure 5.3 give the result. In both dimensions the share rises monotonically with node count for every layout, and at 21 of the 22 size–node points it is strictly ordered by MPI rank count: fewer ranks, less exposure. Halo exchange is the larger term throughout. In 2D at 165m on 32 nodes it accounts for 34.3 of flat MPI’s 48.2 percentage points, against 13.9 for the reductions. In 3D at the same point it accounts for 70.9 of 84.6. The 3D shares are far higher than the 2D shares at every comparable point, which is consistent with the seven-point stencil’s larger surface-to-volume ratio.

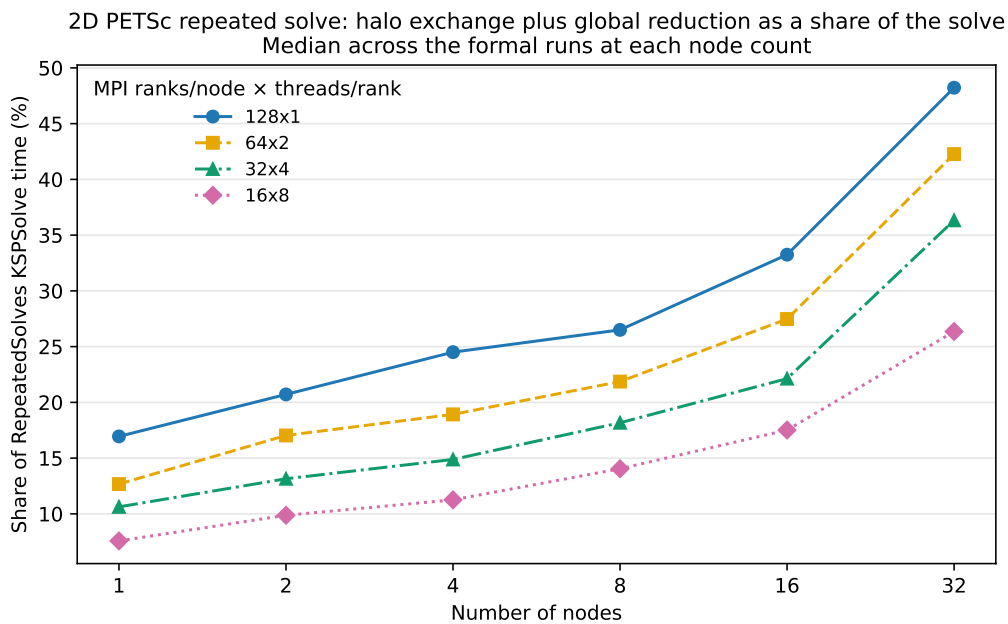


Figure 5.2: 2D share of `RepeatedSolves` `KSPSolve` time spent inside halo exchange and global reduction events. The ordering matches the time-per-iteration ordering in Figure 4.1 at every node count.

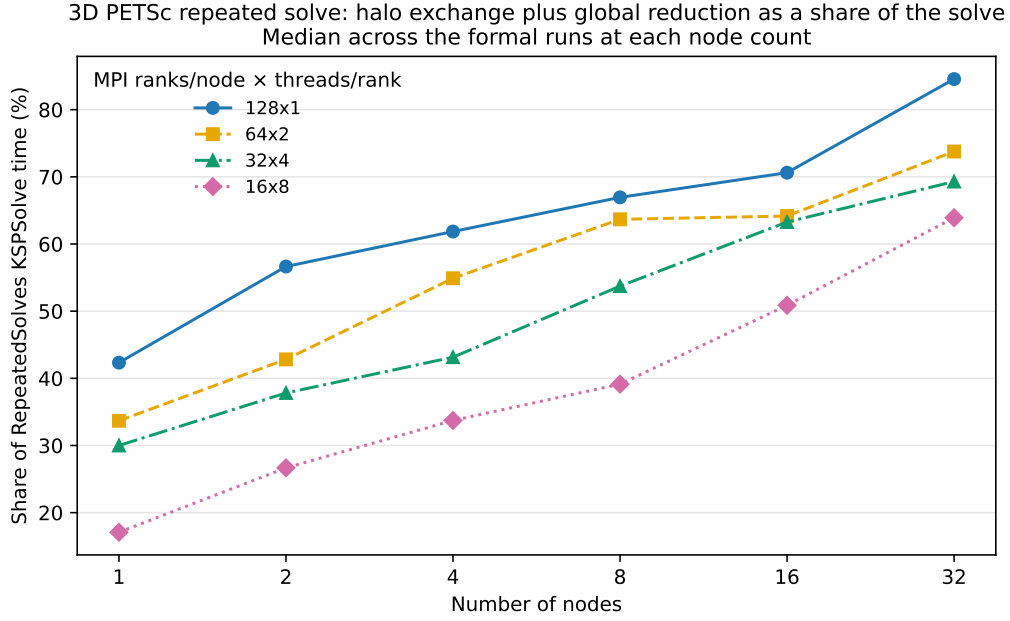


Figure 5.3: 3D share of `RepeatedSolves KSPSolve` time spent inside halo exchange and global reduction events. Note the vertical scale: flat MPI reaches 84.6% at the largest point.

5.4.3 Reduced Exposure is Necessary but not Sufficient

The ordering alone cannot explain the results, because 16×8 has the lowest exposure at nearly every point and the worst time per iteration at all 11 points in 2D. What does track performance is the *size* of the reduction. Within a single layout, comparing its exposure against flat MPI at the same size–node point with its time per iteration relative to flat MPI gives the fits in Table 5.3.

Table 5.3: Reduction in synchronisation exposure relative to flat MPI at the same size–node point, against time per iteration relative to flat MPI. The break-even column is where the fitted line crosses parity, i.e. how much exposure a layout must remove before its thread overhead is repaid. Eleven points per layout; the fits are descriptive.

Case	Layout	Exposure removed (pt)	Time ratio	r	Break-even (pt)
2D	64×2	0.8–7.4	0.872–0.995	−0.67	1.2
2D	32×4	5.3–13.7	0.937–1.096	−0.59	11.7
2D	16×8	6.3–21.9	1.294–1.674	+0.00	not reached
3D	64×2	1.9–15.1	0.768–1.022	−0.86	2.9
3D	32×4	5.5–23.5	0.751–1.070	−0.90	7.3
3D	16×8	17.4–31.1	0.899–1.122	−0.83	26.7

Figure 5.4 plots the 3D case that the table summarises. The three layouts form three roughly parallel descending lines, displaced from each other along the horizontal axis: each removes more exposure than the one before it, and each needs more of it before crossing parity.

Five of the six correlations are negative and three exceed 0.8 in magnitude: the more exposure a layout removes at a given point, the faster it is relative to flat MPI there. The break-even column quantifies the trade-off that Chapter 4 could only assert. Each doubling of threads per rank removes more exposure but demands more before it pays: 1.2, then 11.7, then unreachable in 2D; 2.9, then 7.3, then 26.7 in 3D. No break-even figure is an extrapolation: every one of them,

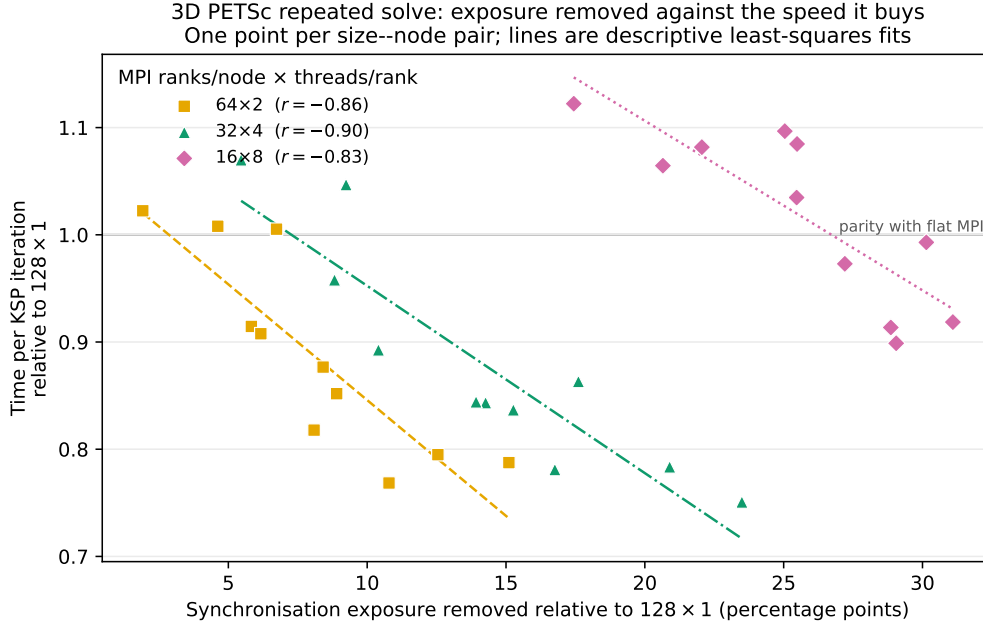


Figure 5.4: 3D exposure removed relative to flat MPI against the resulting time per iteration, one point per size--node pair. Where a layout's fitted line crosses parity is the break-even column of Table 5.3; the lines are descriptive, not predictive.

where it exists, falls inside the range of exposure the layout was actually observed to remove.

This explains the two things Ch5 could not. In 3D at 41m on four nodes, 32×4 removes 23.5 points and is the fastest measured configuration at 0.751. Four nodes later, at 41m on eight nodes, 64×2 removes only 2.0 points and loses at 1.022. The 3D advantage is non-monotone because the exposure it removes is non-monotone, not because the mechanism differs. In 2D, 32×4 crosses parity between eight and 16 nodes precisely as its removed exposure passes its 11.7-point break-even: 9.6 points at 41m on eight nodes for a ratio of 1.003, and 13.7 points at 82m on 16 nodes for 0.944.

The 2D 16×8 row is the counter-example, and Figure 5.5 shows it as a flat cloud among two descending lines. Its correlation is zero: exposure removal predicts nothing there, because the layout is slower than flat MPI by 29–67% at every point and some other cost dominates entirely.

Table 5.2 identifies that cost: at the same problem size and node counts, 16×8 spends more of its core-time waiting in the OpenMP runtime than any other layout, 76.5% against 48.3% for 64×2 at eight nodes. The two halves of the trade are therefore both measured, one on the full matrix without a profiler and the other on eleven profiled cells with one. What remains unmeasured is their sum: the profiled and unprofiled quantities come from different instruments on different runs and cannot be added into a single budget.

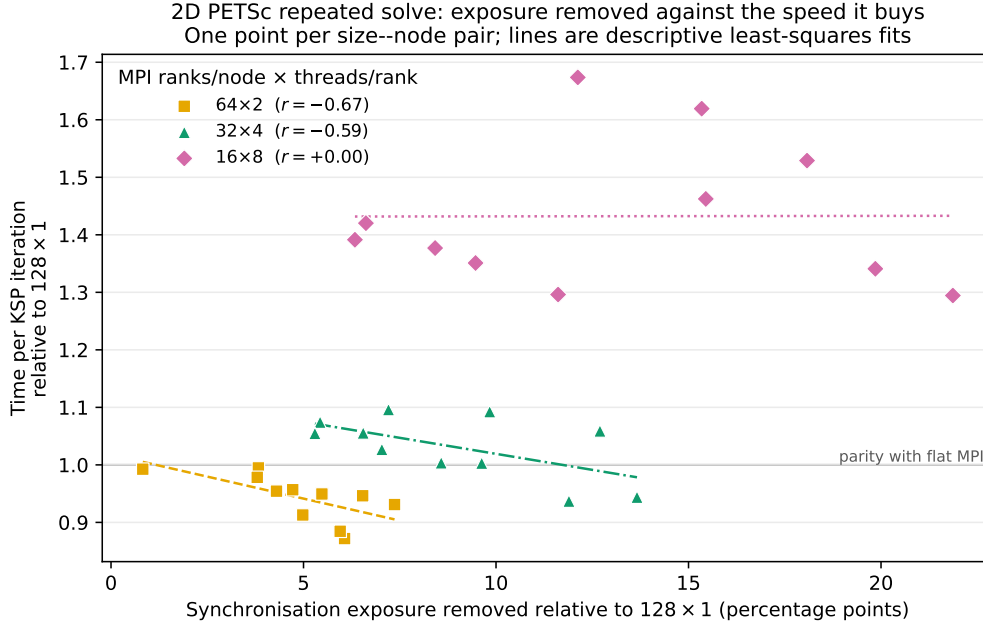


Figure 5.5: 2D exposure removed relative to flat MPI against the resulting time per iteration. Compare with Figure 5.4: 64 $\times$ 2 and 32 $\times$ 4 descend as they do in 3D, but 16 $\times$ 8 sits well above parity at every point with no slope at all.

5.4.4 What This Does and Does Not Establish

Pooling all 33 hybrid measurements per dimension inverts the sign of the correlation (-0.58 in 2D), because 16 $\times$ 8 combines the largest exposure reduction with the worst performance. The relationship is therefore a within-layout one, and reporting a pooled figure would be misleading.

These data identify *where* the layout advantage comes from within PETSc's own event accounting: reduced exposure to halo exchange, with global reduction second and much less layout-sensitive. Together with Section 5.3 they locate the countervailing cost in the OpenMP runtime rather than at a memory boundary. Neither identifies a *hardware* mechanism. The exposure share cannot separate message latency, bandwidth, rank-count-dependent metadata, and waiting caused by load imbalance elsewhere in the iteration. Knowing that threads wait does not say whether they wait because PETSc threads only part of the path, because the threaded regions are too short to amortise a barrier, or because memory bandwidth is already saturated. Separating those, and attributing anything to cache capacity, needs the hardware-counter work listed in Section 7.2.

Chapter 6

Discussion

6.1 Answering the Research Questions

The four questions of Section 1.2 are answered in turn. RQ1 and RQ2 rest on Chapter 4, RQ3 on Chapter 5, and RQ4 on the limits of both.

6.1.1 RQ1: Does the threaded backend reduce time to solution, and by how much?

Hybrid execution reduces steady-state solve time in both dimensions, but the two cases answer the question differently, and the difference is in the shape of the result rather than in which layout happens to win.

In 2D the answer is ordered. Along both unknowns/core chains, every hybrid layout improves relative to flat MPI as the node count grows, and the improvement is monotone in four of the six measured series. 64×2 has a lower time per iteration than flat MPI at all 11 size-node points, and its margin widens from 4.3% at one node to 12.8% at 16 nodes. 32×4 crosses from 9.6% slower at one node to 6.3% faster at 32. A layout comparison performed at small node counts therefore does not transfer to large ones, which is the practical content of the 2D result.

In 3D the effect is larger but unordered. The best hybrid margin is 33.2%, against 14.7% in 2D, and two-thirds of the hybrid measurements are faster than flat MPI. No series is monotone: the advantage peaks at two to four nodes, disappears by eight to 16, and returns at the largest point. Raw throughput selects 32×4 at six of 11 points, 64×2 at four, and flat MPI at one. The preferred balance depends on problem size and node count in a way that the 2D data do not.

6.1.2 RQ2: At which balance, and how does the answer change with the regime?

At constant work per core the useful range is two to four threads per rank in both dimensions, and the preferred member of that range is dimension-dependent: 64×2 everywhere in 2D, 32×4 or 64×2 in 3D depending on size and node count. Held at a fixed problem size instead, the same question has a much larger answer. On 20.25M unknowns in 2D the four layouts stay within a factor of 1.81 of each other from one to 32 nodes and all are still scaling at the end. On the matched 20.35M 3D problem they diverge: every layout eventually saturates, but the node count at which it does rises with the thread count: eight nodes for flat MPI, 16 for 64×2 and 32×4 , beyond 32 for 16×8 . Flat MPI is $2.6\times$ slower at 32 nodes than at its own optimum, so the gap to 16×8 at 4096 cores reaches $3.81\times$. The strong-scaling regime does not change which layouts are good at moderate scale, but it changes the stakes from single-digit percentages to factors, because what differs between layouts is where they stop scaling.

16×8 wins no raw-throughput point in either weak-scaling campaign, yet it has the best weak-scaling efficiency of all four layouts in 2D. The two statements are compatible: weak-scaling

efficiency measures a layout against its own baseline, and 16×8 degrades least because its per-iteration cost is already dominated by a term that does not grow with concurrency. On its own that is a caution about the metric rather than a recommendation. The fixed-size curves of Section 4.1.2 turn it into one for a specific case. Driven past the admission window to about 4.9 thousand unknowns per core, 16×8 is the fastest 3D layout at 32 nodes and the only one of the four that has not saturated. It is $3.81\times$ faster there than flat MPI, which by then has been getting slower for two doublings. Eight threads per rank buys a later saturation limit, not a better operating point.

That contrast is also a methodological result. The same eleven size–node pairs, read at constant work per core, place the useful range at two to four threads and rank 16×8 last. The same solver and build on a fixed problem make 16×8 the only 3D layout still improving at 4096 cores. A layout recommendation is meaningless without the scaling regime it was measured in.

Raw throughput includes changes in GAMG convergence, whereas time per iteration controls for the outer KSP count. The normalised metric does not make the GAMG hierarchy or communication volume identical across decompositions. Both views are needed: raw throughput measures the time paid by this solver configuration, while time per iteration separates the convergence-count effect.

6.1.3 RQ3: Which parts of the solve account for the difference?

On where the time goes, the answer is now partly positive. Section 5.4 shows that hybridisation reduces the share of the solve exposed to halo exchange and global reduction at 21 of the 22 size–node points, that halo exchange is the larger term throughout, and that within each layout the size of that reduction correlates with how much faster the layout is than flat MPI ($|r|$ up to 0.90). Each doubling of threads per rank removes more exposure but must remove more before it pays, which is why the useful range is two to four threads and why the 3D ordering is non-monotone.

The second half of that trade is measured too, on eleven Linaro MAP profiles of the 20.25M 2D problem. Within the solve window the share of core-time spent waiting in the OpenMP runtime rises with threads per rank (48.3%, 69.3% and 76.5% at eight nodes) while the MPI share falls (46.7%, 22.6% and 6.2%). Both orderings reproduce across two different PETSc builds. This locates the cost that hybridisation must repay in the OpenMP runtime rather than at a memory boundary, and it explains why 16×8 removes the most exposure in 2D and is still the slowest layout at every point. Where the cost is paid is not why it is paid; Section 6.1.4 takes that up.

6.1.4 RQ4: How far do the trends generalise, and what would establish a mechanism?

What remains unexplained is one level lower. Knowing that threads wait does not say why: partial threading of PETSc’s solve path, threaded regions too short to amortise a barrier, and saturated memory bandwidth all predict the same signature, and the exposure share likewise cannot separate message latency, bandwidth, rank-count-dependent metadata, and imbalance elsewhere in the iteration. The profiled and unprofiled quantities also come from different instruments and cannot be summed into one time budget. No hardware mechanism is established here.

How far the trends themselves generalise is bounded by the scope set out in Section 6.2: two structured-grid Poisson problems, one PETSc version and compiler environment, one CG+GAMG configuration, and solves that reuse a fixed operator. Within that scope the patterns are consistent across two dimensionalities, six problem sizes and two scaling regimes, and the exposure and OpenMP-wait orderings that accompany them reproduce on a second PETSc build. Section 7.2 lists the measurements that would extend them: hardware counters for the mechanism, a second

and third fixed problem size for the saturation ordering, and node counts beyond 32 to locate the turning points this campaign brackets only from below.

6.2 Threats to Validity

The validated matrix covers two structured-grid Poisson problems, one PETSc version and compiler environment, and one CG+GAMG configuration. The conclusions need not transfer to unstructured matrices, other preconditioners, or solver phases that call dense BLAS. Preconditioner reuse removes setup from the timed stage, so the result applies to applications that solve repeatedly with the same operator rather than end-to-end single solves. That scope limit is not incidental: as Section 3.8 shows, a campaign that timed single solves instead ranked the layouts differently, because setup accounted for a median of 69% of its measured time. An application dominated by one-off solves should expect the setup ranking, not the one reported here.

Three repeats quantify the observed range but are insufficient for strong significance claims or a detailed noise model. Within the formal matrix the unknowns-per-core filter gives at most two node counts per problem size, so every strong-scaling step there covers one doubling. Reading the same points as two weak-scaling chains recovers six- and five-point curves, but does not add measurements: the chains and the doublings describe one campaign of 11 size-node pairs, and a trend seen along a chain is supported by those points alone. The fixed-size extension of Section 4.1.2 supplies genuine six-point curves, but at one problem size per dimension. Whether the 3D saturation ordering holds at other sizes is untested, and its 32-node point sits outside the admission window by design. Its three-repeat spreads are also larger, up to 11.1%, because the per-solve time at the largest node counts is only a few tens of milliseconds.

Weak-scaling efficiency is computed against each layout’s own single-node time, so it measures degradation and not speed, and it rewards a layout whose baseline is poor. It must be read together with Figure 4.1 and Figure 4.3, never on its own.

Iteration normalisation controls the 9–13 iterations in 2D and 6–7 in 3D, but not decomposition-dependent GAMG work within an iteration. The single-node configuration-space study used the default rank-dependent block-Jacobi solver and provides boundary evidence rather than production layout rankings. It is further limited by a defect established in Section 3.3: its job script’s double-dashed `--ksp_rtol 1e-5` was never consumed by PETSc, so those runs converged against a mesh-dependent tolerance that tightens as $1/N$, reaching 2.4×10^{-10} at 6400^2 . The exploratory campaigns therefore differ from the production ones in both solver and tolerance, and no default-versus-GAMG performance comparison is drawn from them. The conclusions that do rest on that sweep hold at a fixed rank count or per iteration, where the tolerance is common to the runs being compared. All formal runs pass the option with a single dash.

The formal runs used a binary linked to threaded LibSci. Section 5.3 now shows directly that the repeated sparse solve barely calls it, at most 2.1% of solve-window core-time, so LibSci is excluded as an explanation rather than merely unproven as one. MAP timings remain excluded from the scaling curves because profiler overhead is large and uneven, and for the same reason the profile shares are read as an ordering between layouts and not as absolute costs. There is one profile per cell and none at 128×1 . Hardware-mechanism claims remain outside the evidence established here.

6.3 Practical Guidance for PETSc on ARCHER2

For this repeated 2D CG+GAMG workload, 64×2 is the best default among the tested full-node layouts, and the case for it strengthens with job size: it is faster per iteration than flat MPI at every measured point, by 4.3% at one node and 12.8% at 16. Small one-node jobs should still test flat MPI, which gives 5.9% and 9.5% higher raw throughput than 64×2 at 5m and 10m

because it converges in one fewer GAMG iteration. Jobs above roughly eight nodes should also test 32×4 , which is slower than flat MPI below that point and faster above it.

For the matched 3D workload, test both 32×4 and 64×2 at the intended node count, and do not extrapolate from a smaller one. 32×4 wins more points across the matrix and gives the single largest margin, 33.2% at 41m on four nodes, but 64×2 is the better choice at 165m on 32 nodes, where it exceeds flat MPI by 30.1%. On the $\sim 80k$ chain at 16 nodes no hybrid layout beats flat MPI at all.

The 16×8 layout wins no raw-throughput point in either dimension at constant work per core, so eight threads per rank should not be a default. Its favourable weak-scaling efficiency in 2D is not evidence for it, for the reason given in Section 6.2. There is one case where it should be tried: a fixed 3D problem pushed to the point where the other layouts have stopped scaling. At about 5 thousand unknowns per core on 32 nodes it was the fastest layout measured and the only one still improving, while flat MPI had been getting slower for two doublings and both intermediate layouts for one.

The general form of that advice is worth stating separately, because it is the guidance most likely to be applied wrongly. Choose the layout for the regime the job actually runs in. If the work per core is comfortable, above roughly 25 thousand unknowns, the layouts differ by single-digit percentages and 64×2 is a safe default in 2D, with 32×4 worth testing in 3D. If instead the problem is fixed and the node count is being pushed for time to solution, the choice is worth factors, and it is the most-threaded layouts that keep scaling. A benchmark run at one node, or at comfortable occupancy, does not answer the second question.

Chapter 7

Conclusion and Future Work

7.1 Conclusions

The corrected 2D and 3D campaigns contain 264 formal runs across 88 configurations from one to 32 ARCHER2 nodes, and the matched fixed-size extension adds 144 more across 48 configurations. Sixteen pilot logs are preserved but excluded from the statistics. Every formal run uses one warm-up solve and 19 timed solves with the GAMG hierarchy reused, and every one was validated against the build, layout, convergence and library that PETSc itself reports.

At constant work per core, moderate hybridisation provides the most reliable range in both dimensions and eight threads per rank is not competitive in either. Beyond that, the two cases differ in kind.

The 2D layout effect is ordered by concurrency. Read as weak scaling at fixed unknowns/core, every hybrid layout improves relative to flat MPI as the node count grows, monotonically in four of the six measured series. 64×2 is faster per iteration than flat MPI at all 11 size-node points, by 4.3% at one node and 12.8% at 16, and at 164.56M unknowns on 32 nodes it reaches 794.8 million equations per second, 13.1% above flat MPI on raw throughput. 32×4 crosses from 9.6% slower at one node to 6.3% faster at 32.

The 3D effect is larger but unordered. The best margin is 33.2%, at 41m on four nodes, and two-thirds of the hybrid measurements beat flat MPI, but no series is monotone and the preferred layout changes with scale and node count. At the matched largest point, 64×2 reaches 510.4 million equations per second, 30.1% above flat MPI, while one node later on the other chain no hybrid layout wins at all.

Held at a fixed problem size instead of fixed work per core, the same layouts separate far further. Over one to 32 nodes the 2D curves stay within a factor of 1.81 of each other and all four are still improving at 4096 cores. The matched 3D curves diverge, because each layout saturates and turns upward at a node count that rises with its thread count: eight nodes for flat MPI, 16 for 64×2 and 32×4 , beyond 32 for 16×8 . Flat MPI is $2.6\times$ slower again by 32 nodes, where 16×8 is the fastest of the four by a factor of 3.81 over it, having been last in every weak-scaling comparison. What distinguishes the layouts under strong scaling is not their speed but where they stop scaling, so a layout recommendation without its scaling regime is not a recommendation.

Profiling locates both sides of the trade. PETSc's own event data show hybridisation reducing exposure to halo exchange and global reduction. Eleven Linaro MAP profiles show the cost that must be repaid: within the solve window the share of core-time spent waiting in the OpenMP runtime rises with threads per rank, 48.3% to 76.5% at eight nodes, reproducibly across two PETSc builds. The same profiles rule out the linked threaded BLAS library as an explanation, at most 2.1% of solve-window core-time.

These measurements replace the invalid launcher-era layout rankings. They support 64×2 as the default for repeated 2D CG+GAMG solves, with the margin growing rather than shrinking

at scale; a 32×4 versus 64×2 comparison performed at the target node count in 3D; and the most-threaded layouts where a fixed problem is pushed past the point at which the others saturate. Flat MPI remains competitive for small 2D jobs and for selected 3D decompositions. The current evidence establishes these performance patterns and names the software cost behind them, but not the hardware mechanism underneath it.

7.2 Future Work

The MAP profiles establish that the threads wait. Hardware counters are needed to establish why. Memory-bandwidth and cache-miss counters, taken per layout at a fixed size and node count, would separate the three candidate causes that the profiles cannot: PETSc threading only part of the solve path, threaded regions too short to amortise an OpenMP barrier, and bandwidth saturation before all cores are busy. Instrumenting PETSc’s threaded regions directly, rather than sampling them, would give the first of those without a counter at all.

The profiled matrix is also thin. It covers one problem size, two node counts on the production build, three hybrid layouts, and one run per cell, and MAP cannot produce the flat-MPI cell that would anchor the trend. Repeating it at a second size, and at the node counts where the 3D fixed-size curves turn upward, would test whether the OpenMP wait share predicts the saturation point rather than merely accompanying it.

Two extensions follow from the fixed-size results. A second and third fixed problem size would show whether the 3D saturation ordering is a property of the layouts or of 20M unknowns, and node counts beyond 32 would locate the turning point of 32×4 and 16×8 , which this campaign brackets only from below. More repeats at close layout pairs would support confidence intervals in place of min–max ranges. Further experiments should include preconditioner setup, changing operators, unstructured matrices, other compiler environments, and GPU execution.

Bibliography

- [1] Satish Balay et al. *PETSc/TAO Users Manual, Revision 3.24*. Tech. rep. ANL-21/39 Rev. 3.24. Argonne National Laboratory, 2025. DOI: 10.2172/2998643.
- [2] Satish Balay et al. “Efficient Management of Parallelism in Object-Oriented Numerical Software Libraries”. In: *Modern Software Tools in Scientific Computing*. Ed. by Eirik Arge, Are Magnus Bruaset, and Hans Petter Langtangen. Boston: Birkhäuser, 1997, pp. 163–202. DOI: 10.1007/978-1-4612-1986-6_8.
- [3] PETSc Development Team. *Changes: 3.22*. 2024. URL: <https://petsc.org/release/changes/322/> (visited on 07/22/2026).
- [4] MPI Forum. *MPI: A Message-Passing Interface Standard, Version 4.1*. Nov. 2023. URL: <https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf> (visited on 07/22/2026).
- [5] OpenMP Architecture Review Board. *OpenMP Application Programming Interface, Version 5.2*. Nov. 2021. URL: <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5-2.pdf> (visited on 07/22/2026).
- [6] Holly Judge. “Effective Use of MPI+OpenMP on a Cray EX Supercomputer”. In: *Proceedings of the Cray User Group 2022*. 2022. URL: https://cug.org/proceedings/cug2022_proceedings/includes/files/pap104s2-file1.pdf (visited on 07/22/2026).
- [7] Michael Lange et al. “Achieving Efficient Strong Scaling with PETSc Using Hybrid MPI/OpenMP Optimisation”. In: *Euro-Par 2012: Parallel Processing Workshops*. Ed. by Julian M. Kunkel, Thomas Ludwig, and Hans Werner Meuer. Berlin, Heidelberg: Springer, 2013, pp. 11–20. DOI: 10.1007/978-3-642-38750-0_8.
- [8] ARCHER2 Service. *ARCHER2 Hardware*. 2026. URL: <https://docs.archer2.ac.uk/user-guide/hardware/> (visited on 07/22/2026).
- [9] ARCHER2 Service. *Performance Tuning*. 2026. URL: <https://docs.archer2.ac.uk/user-guide/tuning/> (visited on 07/22/2026).
- [10] Magnus R. Hestenes and Eduard Stiefel. “Methods of Conjugate Gradients for Solving Linear Systems”. In: *Journal of Research of the National Bureau of Standards* 49.6 (1952), pp. 409–436. DOI: 10.6028/jres.049.044.
- [11] Klaus Stüben. “A Review of Algebraic Multigrid”. In: *Journal of Computational and Applied Mathematics* 128.1–2 (2001), pp. 281–309. DOI: 10.1016/S0377-0427(00)00516-1.
- [12] Linaro Limited. *Linaro MAP*. Linaro Forge 25.0.4 documentation. 2025. URL: https://docs.linaroforge.com/25.0.4/html/forge/forge/introduction_to_forge/map.html (visited on 07/22/2026).

Appendix A

Reproducibility: Repository and Scripts

The repository accompanying this dissertation contains the benchmark sources, the ARCHER2 job scripts, every raw log, and the analysis pipeline that turns one into the other.

A.1 Layout

Table A.1 gives the directory layout of the repository.

Table A.1: Repository layout. Raw logs and retrieved job CSVs are immutable evidence; everything under a *derived* heading is regenerated by the scripts.

Path	Contents
<code>src/</code>	Benchmark drivers (Section 3.2)
<code>scripts/ksp/{2d,3d}/lib/</code>	Slurm job scripts and problem-size tables
<code>scripts/petsc_configure_archer2.sh</code>	PETSc configure recipes for both builds
<code>analysis/data/<campaign>/logs/</code>	Raw <code>-log_view</code> output, immutable
<code>analysis/data/<campaign>/csv/source/</code>	Retrieved job CSVs, byte-for-byte
<code>analysis/maps/<campaign>/</code>	Linaro MAP profiles, immutable
<code>analysis/data/<campaign>/csv/derived/</code>	Derived: normalised and aggregated CSVs
<code>analysis/data/<campaign>/tables/</code>	Derived: reference tables
<code>analysis/plots/<campaign>/</code>	Derived: figures, as PNG and PDF pairs
<code>analysis/scripts/<campaign>/</code>	Analysis code and its unit tests
<code>docs/dissertation_latex/</code>	This document and <code>sync_figures.sh</code>

The campaigns are named for the study that produced them. The results in this dissertation come from `nproc_2d_libsci` and `nproc_3d_libsci`, which hold both the validated repeated-solve matrices and, under `strong_scaling_20m`, the fixed-size extension of Section 4.1.2. The same 2D campaign also holds the earlier `multi_node`, `single_node_hybrid` and `single_node_pure_mpi` run sets, collected on the same build and driver while the configuration space was being fixed; of these only `multi_node` is used in the text, for Table 3.3. `size_grid` holds the single-node configuration-space sweep of Section 3.5, and `profiling_2d_map_baseline` the baseline MAP profiles of Section 5.3. The earlier `ksp_2d_scaling`, `ksp_3d_scaling`, `mpi_only` and `fixed128_multinode` campaigns are retained as provenance and are not used for any layout claim, for the reasons given in Section 3.3 and Section 6.2.

A.2 Evidence Index

Every numbered figure and table in this dissertation is derived from raw job output by one of six scripts. Table A.2 gives the chain for each: which script produced it, which derived file that script writes, and which raw evidence the script consumed. To check any number, run the script named in its row and compare; to check the number’s provenance instead, read the raw evidence directly, which is retained unmodified.

Table A.2: Evidence index. Script paths are relative to `analysis/scripts/`, derived and raw paths to `analysis/`. Log counts are the files present, which for the 3D fixed-size campaign exceeds the 72 accepted runs because superseded job attempts are retained. The last two rows are read directly from a named file and have no derived stage.

Figures and tables	Generated by	Derived output	Raw evidence
Table 3.1, Table 3.2, Figure 3.1	<code>size_grid/ analyze_single_node_ sweep.py</code>	<code>data/size_grid/tables/ iterations_vs_ layout.csv, fullnode_cost_per_ iteration.csv</code>	<code>data/size_grid/logs/ 213 logs</code>
Table 3.3	<code>nproc_2d_libsci/ analyze_multinode_ thread_limit.py</code>	<code>tables/multinode_ thread_limit.csv</code>	<code>logs/multi_node/ 45 logs, reconciled against csv/derived/fullnode_ runs.csv</code>
Table 4.1, Table 4.4, Figure 4.1, Figure 4.2, Figure 4.3, Figure 4.4, Figure B.1, Figure B.2	<code>nproc_{2d,3d}_libsci/ analyze_repeated_ fullnode_logs.py</code>	<code>csv/derived/repeated_ fullnode_formal_ summary.csv</code>	<code>logs/repeated_ fullnode/ 140 logs per dimension</code>
Table 4.5, Table 4.6, Figure 4.5– Figure 4.7	<code>strong_scaling_20m/ analyze_strong_ scaling_20m.py</code>	<code>csv/derived/strong_ scaling_20m/*.csv, tables/strong_ scaling_20m/*.csv</code>	<code>logs/strong_ scaling_20m/ 72 (2D) and 108 (3D) logs</code>
Table 4.3	<code>cross_dimension/ summarize_best_ layouts.py</code>	<code>tables/weak_scaling_ best_layouts.csv</code>	the two formal summaries above
Table 4.7	<code>cross_dimension/ summarize_cross_ dimension.py</code>	<code>tables/cross_dimension_ summary.csv</code>	joins the three rows above
Table 5.2, Figure 5.1	<code>map_profiles/ analyze_map_ profiles.py</code>	<code>csv/derived/map_ profiles/map_call_tree_ shares.csv</code>	<code>maps/ 12 MAP profiles, 11 usable</code>
Table 5.3, Figure 5.2– Figure 5.5	<code>nproc_2d_libsci/ analyze_event_ breakdown.py (covers both dimensions)</code>	<code>tables/repeated_ fullnode_event_ matrix.csv, repeated_fullnode_ exposure_fits.csv</code>	<code>logs/repeated_ fullnode/ as above</code>
Table 2.2, Table 3.4	read directly	—	<code>scripts/ksp/ diagnostics/ petsc_diagnostics. 14598711.out</code>
Table 2.3, Table 5.1	read directly	—	one named <code>-log_view</code> log each, cited in place

Table 2.1 and Table 4.2 state definitions rather than measurements and have no evidence chain: the first classifies the execution models and the second restates the admission rule of Section 3.4. Figure 2.1 is reproduced from the ARCHER2 hardware documentation.

A.3 Regenerating the results

From the repository root, after creating the analysis environment with `python3 -m venv analysis/.venv` and `analysis/.venv/bin/pip install -r requirements.txt`:

```
# unit tests (30)
analysis/.venv/bin/python -m pytest -c analysis/pytest.ini analysis/scripts

# single-node configuration-space sweep -> Table 3.1, Table 3.2, Figure 3.1
analysis/.venv/bin/python \
    analysis/scripts/size_grid/analyze_single_node_sweep.py

# multi-node threads-per-rank restriction -> Table 3.3
analysis/.venv/bin/python \
    analysis/scripts/nproc_2d_libsci/analyze_multinode_thread_limit.py

# 2D and 3D repeated-solve campaigns -> Chapter 4 weak-scaling figures
analysis/.venv/bin/python \
    analysis/scripts/nproc_2d_libsci/analyze_repeated_fullnode_logs.py
analysis/.venv/bin/python \
    analysis/scripts/nproc_3d_libsci/analyze_repeated_fullnode_logs.py

# matched fixed-20m strong scaling -> Chapter 4 strong-scaling figures
analysis/.venv/bin/python \
    analysis/scripts/strong_scaling_20m/analyze_strong_scaling_20m.py \
    --dimension 2d
analysis/.venv/bin/python \
    analysis/scripts/strong_scaling_20m/analyze_strong_scaling_20m.py \
    --dimension 3d

# duplicate-batch reproducibility check -> Section 4.1.2
analysis/.venv/bin/python \
    analysis/scripts/strong_scaling_20m/compare_superseded_batches.py

# event-level breakdown and exposure fits -> Section 5.4
analysis/.venv/bin/python \
    analysis/scripts/nproc_2d_libsci/analyze_event_breakdown.py

# Linaro MAP call-tree shares -> Section 5.3
analysis/.venv/bin/python \
    analysis/scripts/map_profiles/analyze_map_profiles.py

# weak-scaling best layout per point -> Table 4.3
analysis/.venv/bin/python \
    analysis/scripts/cross_dimension/summarize_best_layouts.py

# cross-dimensional comparison -> Table 4.7
```

```
analysis/.venv/bin/python \  
    analysis/scripts/cross_dimension/summarize_cross_dimension.py  
  
# copy the generated PDFs into this document's figures/ directory  
bash docs/dissertation_latex/sync_figures.sh
```

Each script prints the number of logs it validated and the configurations it aggregated, and exits non-zero if any check fails. The document is then built with `tectonic docs/dissertation_latex/draft.tex`.

Appendix B

Supplementary Figures

Figure B.1 and Figure B.2 give raw repeated-solve throughput for the weak-scaling campaign of Section 4.2. Throughput rises almost linearly with the node count along both chains in both dimensions, which is the result they carry. They cannot carry the layout comparison: on a logarithmic axis the four layouts differ by a few per cent against a range of three orders of magnitude, which is why Figure 4.1 and Figure 4.3 normalise each layout against flat MPI at the same size–node pair instead.

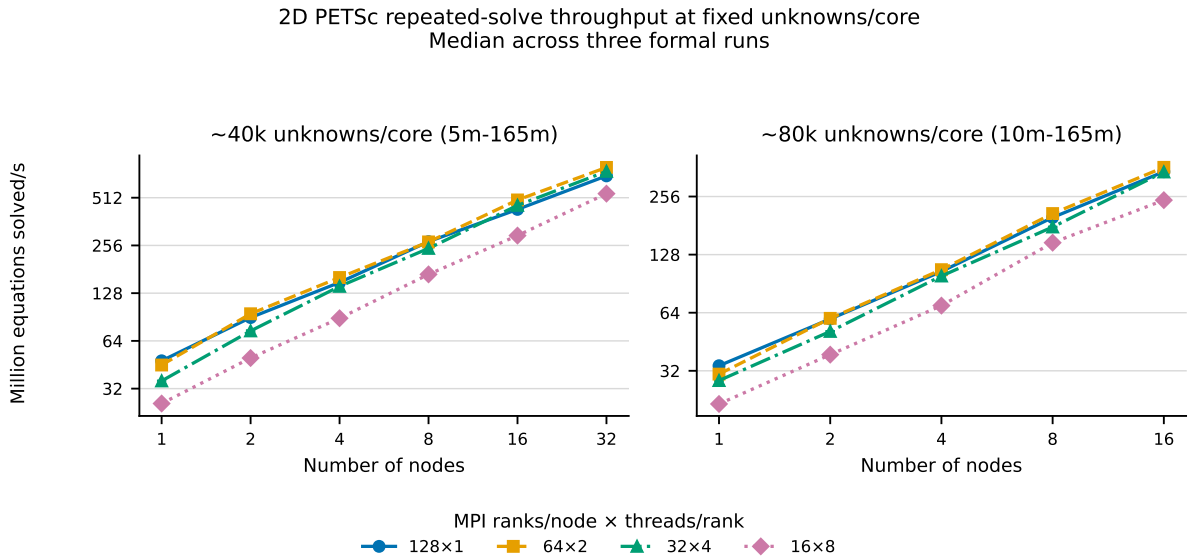


Figure B.1: Median 2D repeated-solve throughput along the two unknowns/core chains. Whiskers show the observed range of three formal runs. All layouts use 128 cores per node, so a fixed node count means a fixed core count.

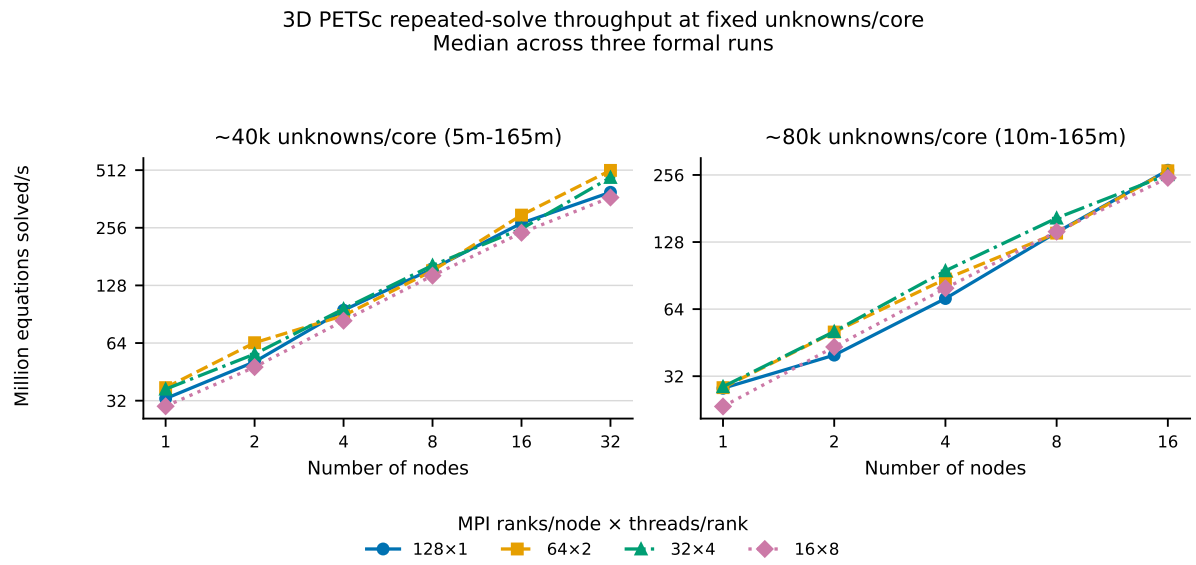


Figure B.2: Median 3D repeated-solve throughput along the two unknowns/core chains. Whiskers show the observed range of three formal runs.

Appendix C

Full log_view Excerpts